A Seminar Report

on

“Intelligent Autoscaling in Cloud-Native Microservices”

Submitted to the

Savitribai Phule Pune University

In partial fulfillment for the award of the Degree of

Bachelor of Engineering

in

Information Technology

by

Kripal Singh Bisht

Roll No: 8329

TE IT-B

Under the guidance of

Dr. Ashwini Sapkal



Department of Information Technology

Army Institute of Technology,

Dighi Hills, Pune - 411 015.

2026-2027



CERTIFICATE

This is to certify that the seminar report entitled **"Intelligent Autoscaling in Cloud-Native Microservices"** being submitted by **Kripal Singh Bisht (Roll No. 8329 & TE IT-B)** is a record of bonafide work carried out by him under the supervision and guidance of **Dr. Ashwini Sapkal** in partial fulfillment of the requirement for **TE (Information Technology Engineering) – 2024 course** of Savitribai Phule Pune University, Pune in the academic year 2026-2027.

Date:

Place: Army Institute of Technology, Pune

Guide Head of the Department

Principal

This Seminar report has been examined by us as per the Savitribai Phule Pune University, Pune requirements at Army Institute of Technology, Pune – 411015 on . . . . . .

Internal Examiner External Examiner

ACKNOWLEDGEMENT

I sincerely thank my seminar guide, **Dr. Ashwini Sapkal**, of the Department of Information Technology, Army Institute of Technology, Pune, for her steady guidance, encouragement and support throughout this seminar. Her suggestions during the reviews, her command of the subject and her patient supervision helped me turn a broad interest in cloud computing into a focused study of intelligent autoscaling.

My thanks also go to the **Head of Department, Prof. Sangeeta Jadhav**, and to all faculty members of the Department of Information Technology for their feedback and cooperation during the seminar reviews, which helped me sharpen the scope of this work and improve this report. I am also grateful to the **Army Institute of Technology** for the supportive academic environment and the facilities that made this work possible.

Finally, I thank my friends and family for their constant encouragement, patience and moral support.

Name – Kripal Singh Bisht

Sign. -

CONTENTS

Sr. No.	TITLE	Page No.
	Abstract	7
1.	**Introduction**	8
1.1	Background	8
1.2	Importance and Context	10
1.3	Autoscaling Mechanisms in Kubernetes	12
1.4	Problem Statement	12
1.5	Organisation of the Report	13
2.	**Literature Survey**	14
2.1	**Summary of the Literature Survey**	22
3.	**Motivation, Purpose, Scope and Objective**	24
3.1	Motivation and Purpose	24
3.2	Scope	24
3.3	Objectives	25
3.4	Challenges and Possible Safeguards	25
4.	**Methodology and Algorithms**	26
4.1	Problem Formulation and Overall Approach	26
4.2	Baseline: Kubernetes Horizontal Pod Autoscaler	27
4.3	Workload Prediction Module	28
4.4	Dependency Analysis Module	29
4.5	Capacity Estimation	30
4.6	Multi-Objective Scaling Policy using Reinforcement Learning	31
4.7	Safety and Constraint Layer	34
4.8	Complete Algorithm	34
4.9	Proposed Methodology in Stages	35
4.10	Evaluation Plan	37
4.11	Improvements of the Proposed Framework over Kubernetes HPA	38
5.	**Discussion and Conclusion**	39
5.1	The Core Problem	39
5.2	Discussion	39
5.3	Expected Impact and Contributions	39
5.4	Limitations	39
5.5	Future Work	40
5.6	Conclusion	40
6.	**References**	41
	Plagiarism Check Report	45

LIST OF FIGURES

Fig. No.	Title	Page No.
1.1	From a monolithic application to cloud-native microservices	8
1.2	Kubernetes cluster running User, Orders and Payments microservices as Pods on three worker nodes	9
1.3	Three ways to add capacity in Kubernetes: horizontal, vertical and node scaling	10
1.4	Service Level Indicators, Objectives and Agreements, and the error budget	10
1.5	Illustrative example (synthetic data): fixed capacity either wastes resources or falls short, while autoscaled capacity follows demand	11
2.1	Evolution of autoscaling approaches for cloud-native microservices	14
2.2	Taxonomy used to classify the surveyed autoscalers	16
2.3	Research gaps identified from the literature survey	17
3.1	Key challenges in autoscaling cloud-native microservices	24
4.1	Proposed predictive, dependency-aware and constrained autoscaling framework	26
4.2	Reactive autoscaling as a MAPE-K feedback loop in the Kubernetes HPA	27
4.3	Illustrative example (synthetic data): reactive capacity lags a demand spike, while forecast-driven capacity rises in time	28
4.4	Service dependency graph of the Online Boutique benchmark (eleven microservices)	30
4.5	Reinforcement-learning formulation of microservice autoscaling	32
4.6	RL training environment with cluster and simulation modes, redrawn after the Gwydion framework [31]	34
4.7	Proposed methodology for predictive, dependency-aware and constrained autoscaling	36

LIST OF TABLES

Table No.	Title	Page No.
1.1	Autoscaling mechanisms available in Kubernetes	12
2.1	Literature survey of twenty-nine studies on intelligent autoscaling	18
2.2	Comparative synthesis of autoscaling approach families	23
3.1	Practical challenges in intelligent autoscaling and possible safeguards	25
4.1	Worked example of dependency-aware capacity estimation (assumed values)	31
4.2	Stage-wise description of the proposed methodology	36
4.3	Evaluation metrics for the proposed autoscaler	37
4.4	Planned experiments	37
4.5	Comparison of Kubernetes HPA and the proposed framework	38

ABSTRACT

Cloud-native applications are now built as sets of small, independently deployable **microservices** that run in containers and are managed by orchestration platforms such as **Kubernetes**. The traffic these applications receive is rarely steady: it follows daily cycles, sudden promotions and unpredictable bursts. **Autoscaling** adjusts the number of running instances (Pods) or the resources given to them so that capacity follows demand. If capacity is too low, response times rise and **Service Level Objectives (SLOs)** are violated; if it is too high, money and energy are wasted.

The problem matters in practice: industry surveys report that 82% of container users now run Kubernetes in production and that 83% of container costs are associated with idle resources, while data centres already use about 1.5% of the world's electricity. Better autoscaling therefore saves money and energy as well as protecting users.

The default Kubernetes **Horizontal Pod Autoscaler (HPA)** is a reactive, threshold-based controller. It is simple and dependable, but it acts only after a change in load has been measured, it looks at each microservice in isolation, and it relies mainly on CPU utilisation. In a chain of dependent services this leads to late reactions, hidden downstream bottlenecks and either over-provisioning or SLO violations during traffic spikes.

This seminar studies **intelligent autoscaling**, in which forecasting, machine learning, **Deep Reinforcement Learning (DRL)**, dependency-aware graph models and safety mechanisms are added to the scaling control loop. Twenty-nine studies published between 2019 and 2026 are surveyed and compared, including production-scale systems at Google and Alibaba, covering predictive models (Random Forest, Bi-LSTM, SCINet), DRL agents (DQN, A2C, PPO, TD3, meta-RL), graph neural networks, hybrid proactive–reactive controllers, multi-layer and energy-aware scaling and joint scaling and placement. The survey shows a clear evolution from single-metric, per-service rules toward **predictive, dependency-aware, multi-objective and guarded** control.

Based on the gaps found, the report proposes a conceptual **Predictive, Dependency-Aware and Constrained Autoscaling Framework**. It collects telemetry, forecasts demand per service, propagates expected load through the service graph, estimates required replicas, selects actions with a multi-objective policy, and passes every action through a safety layer with a fallback to HPA before execution through the Kubernetes API. An evaluation plan against a tuned HPA baseline is also defined. The framework is a research proposal and has not been validated as a production system; all numerical results quoted belong to the original authors.

Keywords: Cloud-native computing, Microservices, Kubernetes, Autoscaling, Horizontal Pod Autoscaler, Workload prediction, Deep Reinforcement Learning, Service Level Objectives.

1. INTRODUCTION

1.1 Background

- **Cloud computing** lets organisations rent processing power, memory and storage over the internet instead of buying and maintaining their own servers. Resources are billed by use, so the amount of capacity kept running directly decides the monthly cost.
- Modern applications are increasingly **cloud-native**. Instead of one large **monolithic** program, the application is split into **microservices** such as user, catalogue, cart, payment and recommendation services. Each microservice is packaged as a **container**, owns a narrow piece of business logic and talks to other services over the network through APIs (Fig. 1.1).
- Microservices bring faster development, independent deployment and fault isolation, but they also create a **distributed system**: a single user request may pass through several services, and each service has its own load, resource profile and performance behaviour.

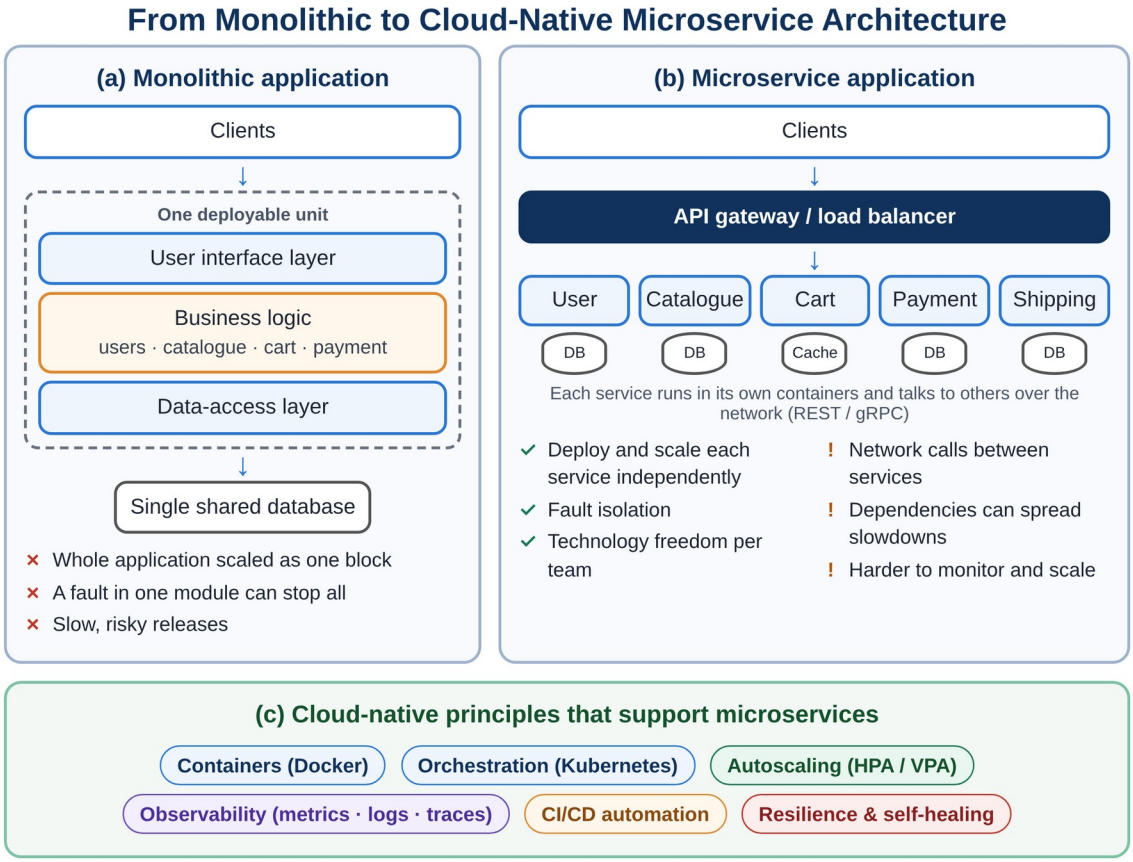


Fig. 1.1: From a monolithic application to cloud-native microservices

- **Kubernetes (K8s)** has become the standard platform for running containers. A cluster has a **control plane** (API server, scheduler, controller manager, etcd) and several **worker nodes**. Containers run inside **Pods**, the smallest deployable unit, and a **Deployment** keeps a desired

number of identical Pod replicas running. A load balancer or Service spreads incoming requests across the replicas (Fig. 1.2).

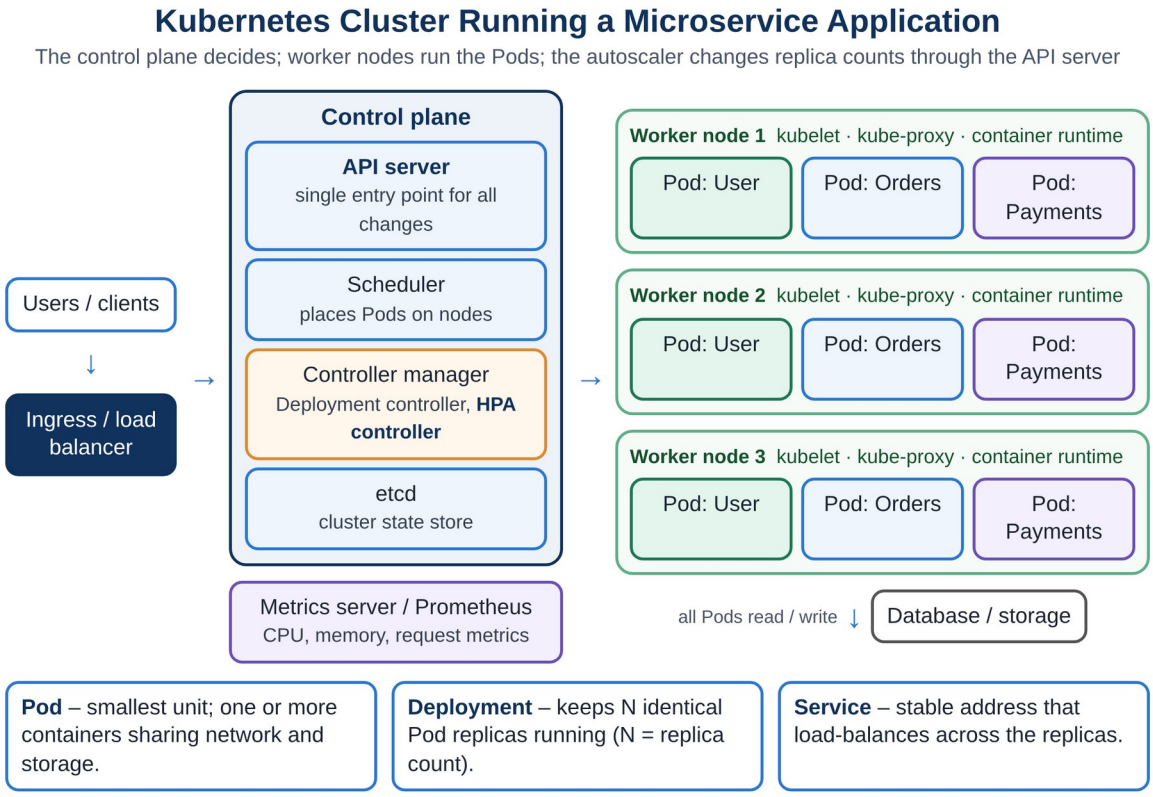


Fig. 1.2: Kubernetes cluster running User, Orders and Payments microservices as Pods on three worker nodes

- **Autoscaling** is the automatic adjustment of capacity to match demand without human intervention. It takes three main forms: **horizontal scaling** (scale out / in) changes the number of Pod replicas; **vertical scaling** (scale up / down) changes the CPU and memory given to each Pod; and **node or cluster scaling** adds or removes worker machines when Pods no longer fit (Fig. 1.3).

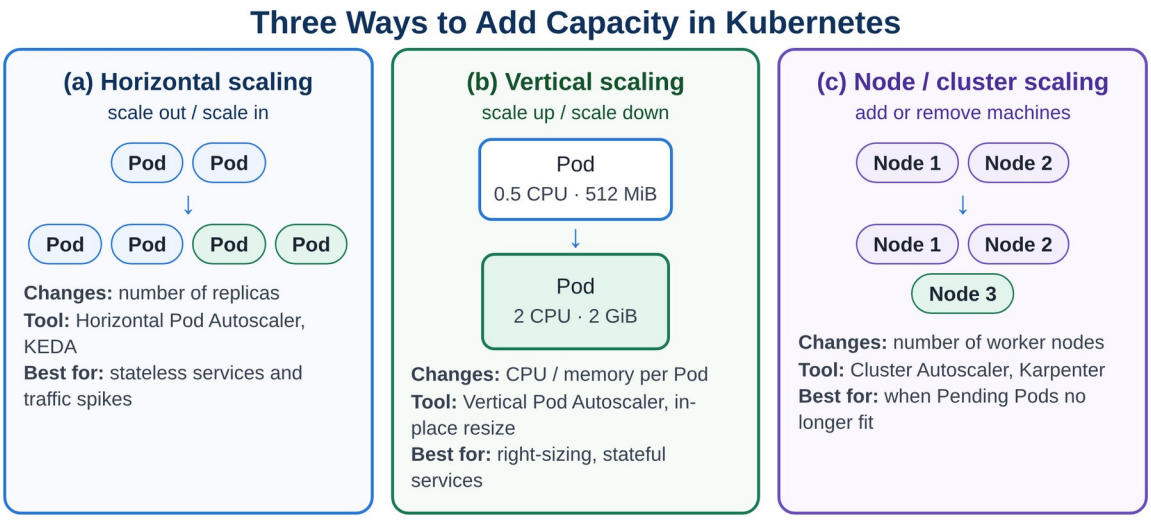


Fig. 1.3: Three ways to add capacity in Kubernetes: horizontal, vertical and node scaling

- The quality of an autoscaler is judged against **Service Level Objectives (SLOs)**: measurable targets set on **Service Level Indicators (SLIs)** such as *"95% of requests answered within 300 ms"* or *"99.9% availability"*. The allowed shortfall is the **error budget**; a 99.9% monthly availability target, for example, allows only about 43 minutes of failure in 30 days (Fig. 1.4).

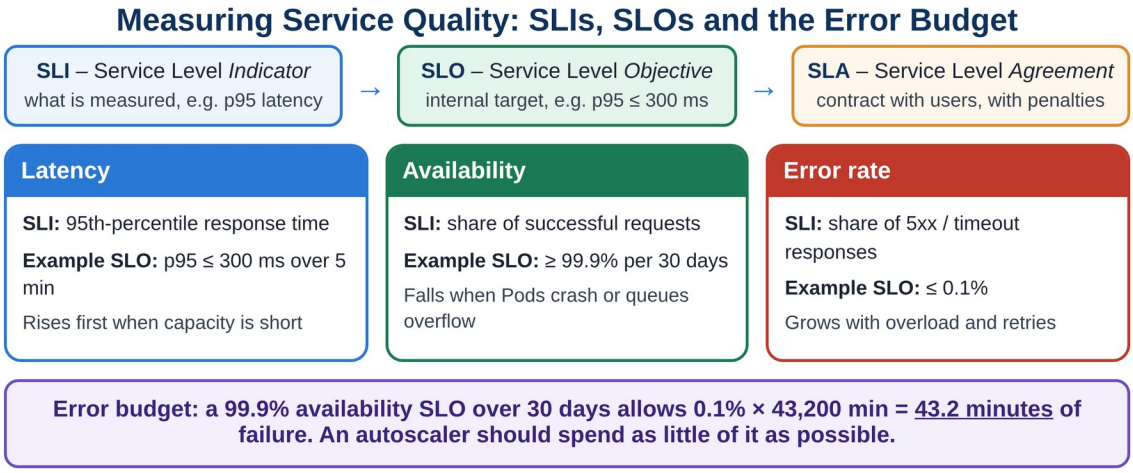


Fig. 1.4: Service Level Indicators, Objectives and Agreements, and the error budget

1.2 Importance and Context

- Cloud workloads are **highly variable**. An e-commerce site may serve a few requests per second at night and thousands during a sale. If capacity is fixed for the peak, most of it sits idle; if it is fixed for the average, the peak causes queues, timeouts and failures. Autoscaling is therefore the main mechanism that links **cost** with **user experience**.

- **Over-provisioning** wastes money and electricity, while **under-provisioning** increases latency, causes SLO violations and loses users (Fig. 1.5).

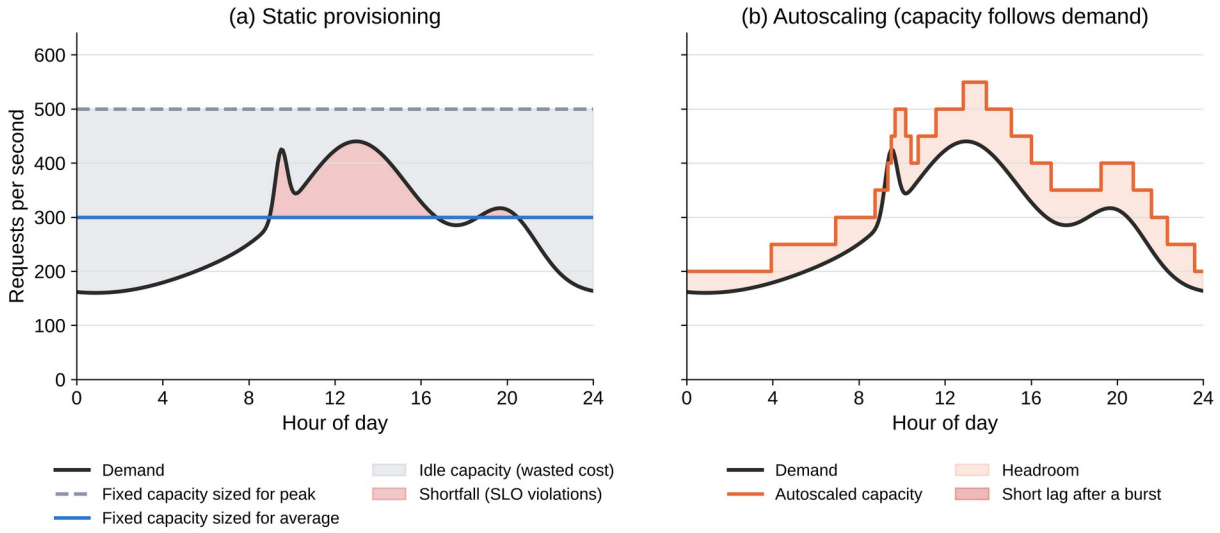


Fig. 1.5: Illustrative example (synthetic data): fixed capacity either wastes resources or falls short, while autoscaled capacity follows demand

- **Kubernetes is now the default platform.** The CNCF Annual Cloud Native Survey published in January 2026 found that 82% of container users run Kubernetes in production, up from 66% in 2023, and that 98% of surveyed organisations use cloud-native techniques [1]. Autoscaling decisions made by Kubernetes therefore affect a very large share of today's online services.
- **Wasted capacity is a real cost.** In Flexera's 2025 State of the Cloud report, 84% of more than 750 respondents named managing cloud spend as their top cloud challenge [2]. Datadog's State of Cloud Costs 2024 study found that 83% of container costs were associated with idle resources, largely because teams over-provision Kubernetes workloads to avoid performance problems [3].
- **Energy and environment.** The International Energy Agency estimates that data centres used about 415 TWh of electricity in 2024, roughly 1.5% of global consumption, and projects this to more than double to about 945 TWh by 2030 [4]. Releasing idle replicas promptly is one of the simplest ways for software to reduce this demand.
- Autoscaling in microservices is harder than in a single application. A slowdown in one service (for example, a product catalogue or a database) can ripple to every service that depends on it, so the service that is overloaded is not always the one that receives the user request.
- The default **Kubernetes HPA** reacts to current metrics, usually CPU utilisation, for each Deployment separately. It is reliable and easy to understand, which is why it is the industry baseline, but it does not anticipate demand and does not know about service dependencies. This gap has motivated research into **intelligent autoscaling** using prediction, machine learning and reinforcement learning.

1.3 Autoscaling Mechanisms in Kubernetes

Kubernetes offers several built-in and community autoscalers, each acting on a different layer (Table 1.1). They are complementary: a typical production cluster combines a workload autoscaler (HPA or KEDA) with a node autoscaler, and uses the VPA or in-place resizing to right-size Pods. All of them are **rule-based**; none forecasts demand or reasons about service dependencies, which is the space that intelligent autoscaling research targets.

Table 1.1: Autoscaling mechanisms available in Kubernetes

Mechanism	What it changes	Signal used	Typical use	Main limitation
Horizontal Pod Autoscaler (HPA) [5]	Replica count of a Deployment	CPU, memory, custom or external metrics	Stateless web and API services	Reactive; per-Deployment; targets tuned by hand
Vertical Pod Autoscaler (VPA) [6]	CPU / memory requests of Pods	Historical usage	Right-sizing requests	Should not act on the same CPU / memory metric as the HPA
In-place Pod resize [7]	CPU / memory of a running Pod without restart	Set by a user or controller	Latency-sensitive and stateful Pods	Stable only since Kubernetes 1.35 (Dec. 2025); limited by node capacity
Cluster Autoscaler / Karpenter [8]	Number of worker nodes	Pending Pods, under-used nodes	Keeping cluster capacity in line	New nodes take minutes to become ready
KEDA [9]	Replicas, including scale to zero	Event sources such as queue length or consumer lag	Event-driven and batch consumers	Still reactive to the current event backlog

1.4 Problem Statement

Reactive autoscalers must first observe a change in load, wait for metrics to be collected, compute a decision and then wait again until new Pods are scheduled, pulled and ready. During this **actuation delay** latency rises. CPU alone may not reveal a queue backlog, a slow database or an overloaded downstream service, and independent per-service scaling can move the bottleneck instead of removing it. At the same time, adding intelligence brings new risks: wrong forecasts, unsafe exploration by learning agents and controllers that are hard to explain.

Problem statement

How can an autoscaling controller for cloud-native microservices use workload history, application telemetry and service dependencies to choose timely, safe and resource-efficient scaling actions that keep latency within the SLO at the lowest practical cost?

1.5 Organisation of the Report

- **Chapter 2** surveys twenty-nine studies on intelligent autoscaling, compares them in a structured table and summarises the problems, traditional approaches, proposed solutions and research gaps.
- **Chapter 3** presents the motivation, purpose, scope and objectives of the seminar, along with the main practical challenges.
- **Chapter 4** describes the methodology and algorithms: the Kubernetes HPA baseline, workload prediction, dependency analysis, the reinforcement-learning policy, the safety layer, the complete algorithm, the staged methodology and the evaluation plan.
- **Chapter 5** discusses the findings, expected contributions, limitations and future work, and concludes the report.

2. LITERATURE SURVEY

This chapter reviews twenty-nine studies on autoscaling for containerised and microservice applications, published between 2019 and 2026, together with recent surveys and two analyses of production microservice traces. The studies are grouped by the kind of intelligence they add to the scaling loop. Fig. 2.1 shows how the field has moved from fixed capacity and simple thresholds toward predictive, learning-based, dependency-aware and guarded controllers. Table 2.1 compares all twenty-nine studies, and Table 2.2 in Section 2.1 groups them into approach families.

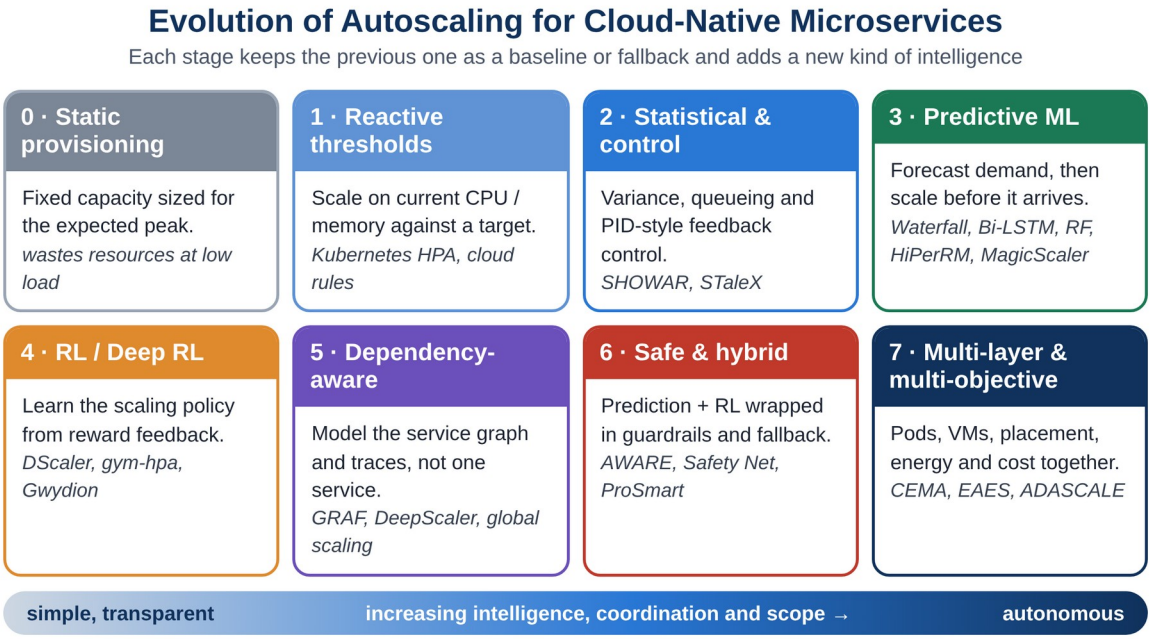


Fig. 2.1: Evolution of autoscaling approaches for cloud-native microservices

What Previous Researchers Have Done

- **Predictive policies and forecasting.** Early work replaced manually chosen thresholds with learned predictive policies. A trace-driven approach built a response-time model from stress tests and learned a scaling policy that met the latency target at only slightly higher cost than reactive scaling [10]. Reinforcement-learning agents were then used to learn the HPA thresholds themselves on Google Kubernetes Engine [11]. The Waterfall autoscaler used regression models (LR, RF, SVR) to predict both the replicas a service needs and the effect of scaling on downstream services [12], and a Bi-LSTM forecaster inside a MAPE loop scaled Kubernetes Deployments before demand arrived [13].
- **Industry-scale autoscaling.** Google's Autopilot automatically sets CPU and memory limits and replica counts for jobs from moving-window usage statistics and a machine-learning recommender. Autopiloted jobs ran with about 23% slack, against 46% for manually managed jobs, and ten times fewer jobs were severely affected by out-of-memory errors; at the time of publication Autopilot managed over 48% of Google's fleet-wide resource usage [14].

- **Statistical control and bursts.** SHOWAR showed that variance-based right-sizing combined with control-theoretic horizontal scaling and scheduling hints can cut tail latency without deep learning [15]. Burst-aware autoscaling added explicit burst detection, because a model that is accurate on average can still miss rare spikes [16].
- **Uncertainty-aware prediction.** Alibaba's MagicScaler combines a multi-scale attentive Gaussian-process forecaster with an uncertainty-aware scaler, so that it can trade running cost against the risk of QoS violations instead of trusting a single point forecast; it outperformed commonly used scalers on three production clusters [17].
- **Deep and meta reinforcement learning.** DScaler learned the horizontal scaling policy directly with deep RL [18]. A meta-RL approach combined periodic workload prediction with a Neural Process so that one policy could adapt to many services, and was deployed at Alipay [19]. gym-hpa connected OpenAI Gym environments to a real Kubernetes cluster so that RL agents could be trained and compared reproducibly [20].
- **Proactive resource sizing.** A Random-Forest autoscaler forecast CPU and memory and used the forecast for both vertical and horizontal scaling, showing that a simple, well-designed model can be operationally strong [21]. HiPerRM used the SCINet time-series model to drive custom VPA and HPA controllers [22].
- **Production readiness and dependency awareness.** AWARE addressed the practical problem that RL agents explore poorly in production, using meta-learning and bootstrapping to adapt safely [23]. GRAF used distributed traces and a graph neural network to decide how much capacity each service in a call graph needs under an SLO [24]. DRPC distributed the decision among TD3 agents to avoid a central bottleneck [25], and Autothrottle separated an application-level learned controller from simple per-service CPU controllers [26].
- **Spatio-temporal graph and control models.** DeepScaler learns the affinity between services and uses an attention-based graph convolutional network to scale interacting services in one step, reporting on average 41% fewer SLA violations at lower cost [27]. STaleX gives each microservice weighted PID controllers whose weights are tuned by a supervisor that uses dependency and workload information, and reports 26.9% lower resource usage than the Kubernetes HPA [28].
- **Global, hybrid and RL frameworks (2025).** Global scaling used architecture-level dependency knowledge with proactive forecasting and reactive correction to avoid the domino effect of local decisions [29]. Smart HPA and ProSmart HPA targeted resource-constrained clusters [30]. Gwydion extended gym-hpa with resource forecasting and evaluated A2C, PPO and recurrent PPO agents whose observation includes every microservice of the application, under cost-aware and latency-aware rewards [31].
- **Multi-layer, energy-aware and interpretable scaling.** CEMA coordinated container scaling with VM scaling and migration on AWS [32]. An energy-aware algorithm priced idle containers and used feedforward and feedback control to release them [33]. HPAQT learned interpretable, SLO-aware thresholds with Q-learning [34].
- **Safety, model selection and placement (2026).** A safety-net approach paired ARIMA, XGBoost and LSTM forecasts with a rollback guardrail and evaluated them on Alibaba traces [35]. A pattern-aware framework used an LLM to pick the forecasting model for each

workload regime [36]. NimbusGuard fed LSTM forecasts into a DQN agent and compared it with HPA and KEDA [37], and ADASCALE coupled scaling with latency-aware placement in cloud-edge clusters [38].

- **What real microservice systems look like.** Alibaba's production traces cover more than 20,000 microservices; their analysis of about 10 billion call graphs showed highly dynamic dependencies and identified the main factors that affect microservice performance [39]. A 2025 follow-up found long-tailed workload and dependency distributions, call graphs that differ from request to request, and strong changes over time, which challenge the assumptions behind many autoscaling studies [40].
- **Surveys and taxonomies.** Recent surveys organise the field by infrastructure, architecture, scaling method, objective and behaviour model [41], by lifecycle and evaluation practice in cloud-native systems [42], by self-adaptation goals and control levels [43], and by machine-learning technique for Kubernetes [44]. A review of DRL for cloud resource scheduling gives the broader RL context [45]. Fig. 2.2 shows the taxonomy used in this report to compare the studies.

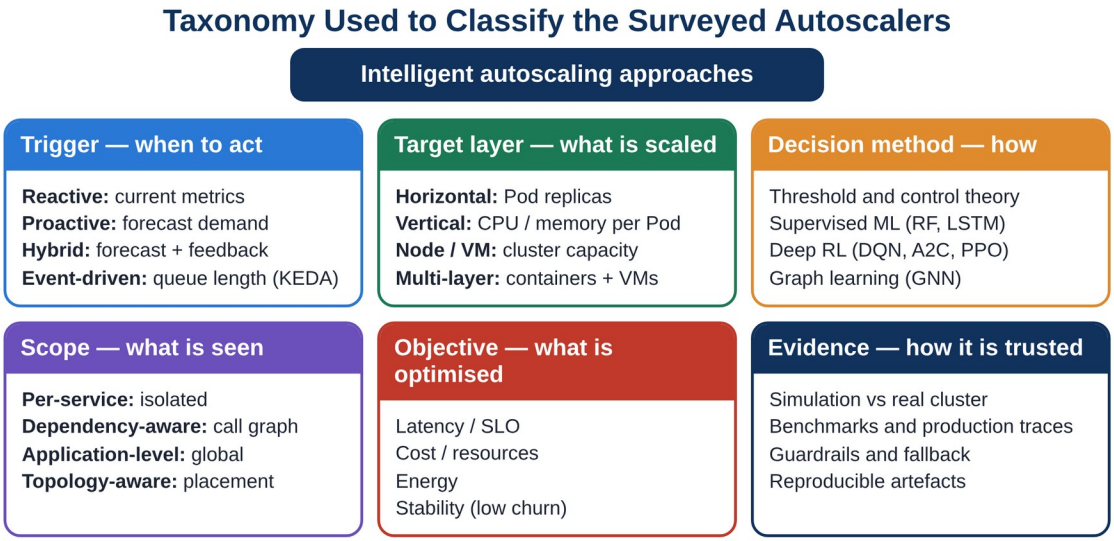


Fig. 2.2: Taxonomy used to classify the surveyed autoscalers

Gaps Identified

The studies above leave six connected gaps, described below and summarised in Fig. 2.3.

- **The cold-start and exploration problem.** RL agents explore before they learn; in a real cluster this exploration causes high latency and SLO violations, and training on a real cluster can take days [20], [23], [31].
- **The dependency blind spot.** Most autoscalers still treat each microservice in isolation, even though a bottleneck in one service affects the whole call chain. Graph-based methods help but are computationally heavy and depend on complete traces [12], [24], [29].

- **Multi-dimensional complexity.** Horizontal, vertical and node scaling are usually optimised separately, and cost, latency, energy and stability are often folded into one weighted reward that hides the trade-off [32], [33], [38].
- **Prediction without uncertainty or safety.** Most predictive systems output a single forecast and act on it directly; only a few model the uncertainty explicitly [17] or add rollback and guardrails [35], [37].
- **Workload drift and unrealistic benchmarks.** Production traces show thousands of services with long-tailed, changing dependencies [39], [40], while most studies evaluate on small benchmarks of ten or fewer services, so models trained on one period or benchmark may not transfer.
- **Weak evaluation standards.** Studies use different benchmarks, workloads, SLO definitions, cost models and baselines, so reported improvements are not directly comparable [41], [42].

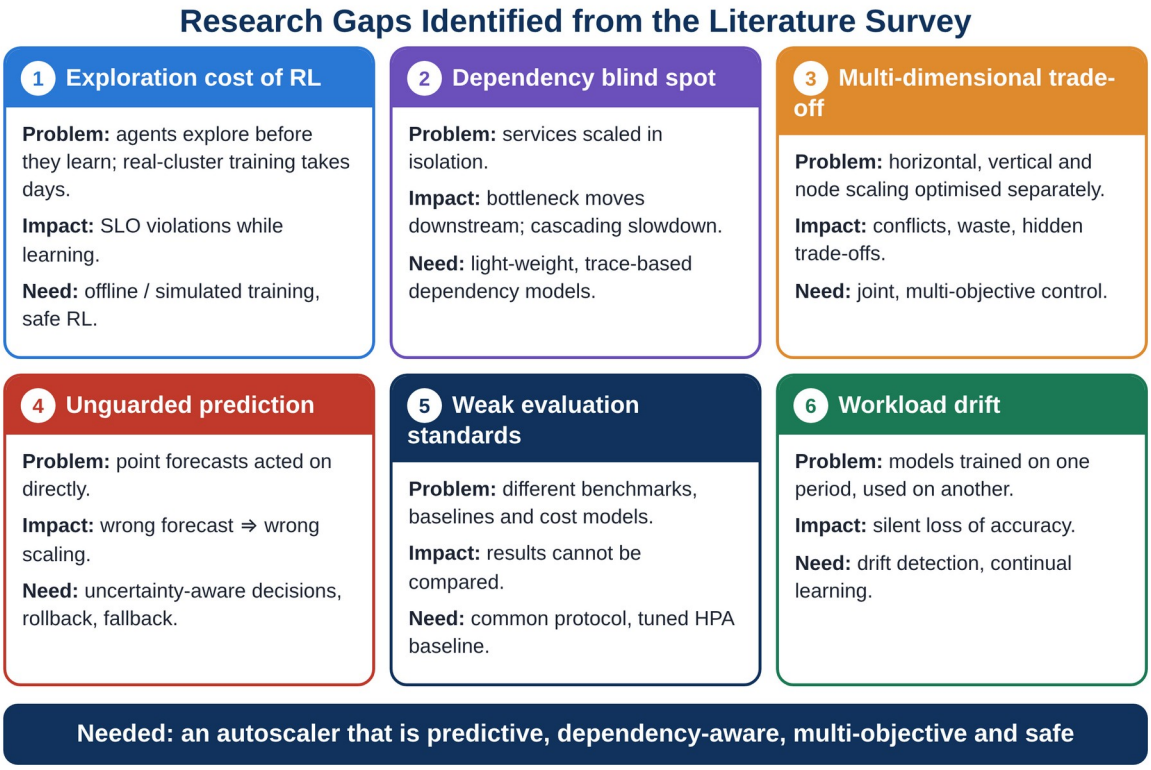


Fig. 2.3: Research gaps identified from the literature survey

Table 2.1: Literature survey of twenty-nine studies on intelligent autoscaling

No.	Paper Title, Journal and Year	Dataset / Environment	Research Focus and Approach	Findings and Strengths	Limitations, Gap and Future Work
1	**Learning Predictive Autoscaling Policies for Cloud-Hosted Microservices Using Trace-Driven Modeling** *IEEE CloudCom, 2019* [10]	Stress-tested microservice; trace-driven simulation; AWS validation	Builds a response-time model from stress tests, then learns a predictive scaling policy in trace-driven simulation	Met the response-time target with only about 4.5% extra cost compared with reactive scaling; reduces data-collection effort	Single-service focus; transfer to other apps and workloads untested. Future: multi-service policies
2	**Autopilot: Workload Autoscaling at Google** *ACM EuroSys, 2020* [14]	Google production fleet (Borg jobs)	Moving-window statistics and an ML recommender set CPU / memory limits and replica counts automatically	Slack cut from 46% (manual) to 23%; 10× fewer jobs severely hit by OOM errors; >48% of fleet usage autopiloted	Designed for Google's jobs; not microservice-graph aware; proprietary
3	**Intelligent Autoscaling of Microservices in the Cloud for Real-Time Applications** *IEEE Access, 2021, Vol. 9* [11]	Google Kubernetes Engine; simulated and real deployments	Resource-demand module plus RL agents that learn HPA threshold values for a response-time QoS	Up to 20% better response time than default autoscaling while keeping QoS	Learns thresholds per service; no multi-service coordination. Future: dependency-aware RL
4	**A Holistic Machine Learning-Based Autoscaling Approach for Microservice Applications** *CLOSER, 2021* [12]	TeaStore benchmark	"Waterfall" autoscaler in a MAPE loop; LR, RF and SVR models predict replicas and the downstream effect of scaling	Lower response time, higher throughput and fewer scaling actions than HPA	Offline models drift; costly data generation; static service graph assumed
5	**Deep Learning-Based Autoscaling Using Bidirectional LSTM for Kubernetes** *Applied Sciences, 2021, Vol. 11(9)* [13]	Kubernetes; realistic web workload traces	MAPE loop with Bi-LSTM forecasting and cooldown / resource-removal rules to curb oscillation	More accurate forecasts than LSTM and ARIMA; outperformed default HPA	Single service; no forecast uncertainty; Pod start-up assumptions. Future: burst handling
6	**SHOWAR: Right-Sizing and Efficient Scheduling of Microservices** *ACM SoCC, 2021* [15]	Several microservice applications; real-world workloads	Variance-based vertical sizing, control-theoretic horizontal scaling and affinity hints to the scheduler	Up to 22% better resource allocation and 20% lower p99 latency	Gains hard to attribute to each part; not learning-based; fast-changing load less explored
7	**Burst-Aware Predictive Autoscaling for Containerized Microservices** *IEEE Trans. Services Computing, 2022, Vol. 15(2)* [16]	Trace-driven simulation; synthetic and real bursty workloads	Combines burst detection, workload forecasting, resource prediction and scaling decisions	More processed requests and fewer SLO violations than the baseline	Sensitive to burst definition and forecast window; cost / SLO trade-off construction
8	**DScaler: A Horizontal Autoscaler of Microservice Based on Deep Reinforcement Learning** *IEEE/IFIP APNOMS, 2022* [18]	Kubernetes microservices; two evaluated workloads	Deep RL learns the horizontal scaling policy directly instead of fixed rules	Lower resource consumption and fewer SLA violations than baselines	Results tied to tested workloads; reward design and training stability. Future: generalisation

No.	Paper Title, Journal and Year	Dataset / Environment	Research Focus and Approach	Findings and Strengths	Limitations, Gap and Future Work
9	**A Meta Reinforcement Learning Approach for Predictive Autoscaling in the Cloud** *ACM KDD, 2022* [19]	Cloud application services; deployed at Alipay	Deep periodic workload predictor plus Neural-Process meta-RL shared across services	Stable CPU utilisation at lower cost; adapts across heterogeneous services without full retraining	Hard to separate gains of prediction and meta-learning; assumes periodic load
10	**gym-hpa: Efficient Auto-Scaling via RL for Complex Microservice-Based Applications in Kubernetes** *IEEE/IFIP NOMS, 2023* [20]	Real Kubernetes cluster; Redis Cluster and Online Boutique	OpenAI Gym environments connected to Kubernetes; RL agents trained with cost and latency rewards	Reproducible RL testbed; agents used fewer resources than default HPA	Real-cluster training is slow; agents can stall on "do nothing". Future: prediction in the loop
11	**Proactive Random-Forest Autoscaler for Microservice Resource Allocation** *IEEE Access, 2023, Vol. 11* [21]	Home-grown microservice platform; real-world workloads	Random Forest forecasts CPU and memory; drives both vertical sizing and horizontal replicas	Up to 90% higher utilisation of allocated resources and 95% better p95 latency	Model update policy; cross-service interactions not modelled
12	**Proactive Resource Autoscaling Scheme Based on SCINet for High-Performance Cloud Computing (HiPerRM)** *IEEE Trans. Cloud Computing, 2023* [22]	Cloud-native Kubernetes cluster experiments	SCINet with RevIN forecasts CPU and memory; custom Hi-VPA and Hi-HPA adjust requests and replicas	Proactive sizing improved resource efficiency over default HPA / VPA	Heavier forecasting model; per-service view; HPA–VPA coordination
13	**MagicScaler: Uncertainty-Aware, Predictive Autoscaling** *PVLDB, 2023, Vol. 16(12)* [17]	Alibaba Cloud ECS; three production clusters	Multi-scale attentive Gaussian-process forecaster plus an uncertainty-aware scaler with stochastic constraints	Outperformed commonly used scalers; explicit cost versus QoS-risk trade-off	Instance-level, not per-microservice; heavier forecasting model
14	**DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning** *arXiv, 2023* [27]	Microservice benchmark applications	Learned affinity matrices and an attention-based graph convolutional network scale interacting services together	On average 41% fewer SLA violations at lower cost	Model complexity; needs history to learn dependencies
15	**AWARE: Automate Workload Autoscaling with Reinforcement Learning in Production Cloud Systems** *USENIX ATC, 2023* [23]	Production-oriented cloud; Gym-like interface	Meta-learning and bootstrapping let RL autoscalers adapt to new workloads safely	Faster adaptation to unseen workloads and fewer SLO violations during training	Needs offline data; behaviour under strong distribution shift
16	**GRAF: Graph Neural Network-Based SLO-Aware Proactive Resource Autoscaling Framework for Microservices** *IEEE/ACM Trans. Networking, 2024* [24]	Kubernetes; open-source microservice benchmarks; large-scale runs	Distributed traces and a GNN estimate traffic impact and choose proactive resources under an SLO	Up to 19% CPU saving over a tuned autoscaler; 36% fewer resources under surge	Graph construction, trace coverage, topology changes and inference overhead

No.	Paper Title, Journal and Year	Dataset / Environment	Research Focus and Approach	Findings and Strengths	Limitations, Gap and Future Work
17	**DRPC: Distributed RL Approach for Scalable Resource Provisioning in Container-Based Clusters** *arXiv preprint, 2024* [25]	Container cluster; realistic testbed and traces	Distributed TD3-based agents decentralise provisioning decisions	Lower latency and fewer failed requests; avoids a central bottleneck	Preprint; agent coordination; production validation pending
18	**Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices** *USENIX NSDI, 2024* [26]	Kubernetes; three microservice apps with production traces	Application-level learning sets targets; per-service heuristic controllers enforce CPU limits	Substantial CPU savings over baselines while meeting latency SLOs	Sensitive to trace characteristics; target propagation semantics
19	**Proactive–Reactive Microservice Architecture Global Scaling** *J. Systems and Software, 2025* [29]	TeaStore and Email Pipeline benchmarks	Architecture-level dependency knowledge plus forecasting and reactive correction	Avoids the domino effect of local scaling; open-science validation	Modelling assumptions; predictor failure; scale of global reconfiguration
20	**Towards Resource-Efficient Reactive and Proactive Auto-Scaling for Microservice Architectures** *J. Systems and Software, 2025, Vol. 225* [30]	Resource-constrained Kubernetes microservices	Smart HPA (reactive resource sharing) and ProSmart HPA (ML-driven proactive policy)	ProSmart HPA reported to outperform Kubernetes HPA; replication package	Specific to studied environment; resource-sharing assumptions
21	**Gwydion: Efficient Auto-Scaling for Complex Containerized Applications in Kubernetes through RL** *J. Network and Computer Applications, 2025, Vol. 234* [31]	Kubernetes cluster and simulation; Redis Cluster and Online Boutique	Gym-based A2C, PPO and recurrent PPO with cost- and latency-aware rewards; observes all microservices; ARIMA / LSTM resource forecasts	Up to 50% lower latency; about 30% cost saving on Online Boutique versus HPA; simulation training far cheaper	Real-cluster training takes days; horizontal scaling only; sim-to-real gap
22	**CEMA: Cost Effective Multi-Layered Autoscaling for Microservice Based Applications** *J. Network and Computer Applications, 2025* [32]	AWS; containers on VMs; real-world workloads	Predictive container scaling guides VM scaling and container migration	More processed requests, fewer SLO violations and lower infrastructure cost	Cross-layer timing; migration cost; node start-up delay
23	**Energy-Aware Elastic Scaling Algorithm for Microservices in Kubernetes Clouds** *J. Network and Computer Applications, 2025* [33]	Kubernetes cloud cluster	SLA-based energy-efficiency model with feedforward and feedback control to release idle containers	Lower cluster energy use while respecting the SLA	Energy-model assumptions; tuning the performance–energy trade-off
24	**HPAQT: Adaptive and Interpretable High-Level SLO-Aware Autoscaling with RL** *ACM UCC, 2025* [34]	Cloud environment	Q-learning adapts HPA thresholds from a high-level SLO; interpretable policy	SLO-aware decisions that operators can inspect	Still threshold-oriented; scaling to many services

No.	Paper Title, Journal and Year	Dataset / Environment	Research Focus and Approach	Findings and Strengths	Limitations, Gap and Future Work
25	**STaleX: A Spatiotemporal-Aware Adaptive Auto-Scaling Framework for Microservices** *arXiv preprint, 2025* [28]	Microservice demo application on Kubernetes	Weighted PID controllers per service, tuned by a supervisor using dependencies and workload changes	26.9% lower resource usage than the Kubernetes HPA	Preprint; controller tuning; small test application
26	**Predictive Autoscaling with a Safety Net for Microservices** *AINA, 2026* [35]	Alibaba microservice traces (20k+ services, 14 days)	ARIMA, XGBoost and LSTM forecasts with a one-minute rollback guardrail; cost model of VM-minutes, SLA violations and churn	Compares reactive, predictive and guarded hybrid policies on a common cost model	Choice of rollback horizon; uncertainty not modelled explicitly
27	**From Reactive to Predictive: A Pattern-Aware Framework for Kubernetes Autoscaling with LLM Integration** *J. Systems and Software, 2026* [36]	Kubernetes	Classifies workload patterns; an LLM selects the forecasting model and hyperparameters	Better forecast accuracy than single-model approaches	LLM reliability, latency and cost; end-to-end scaling benefit unclear
28	**NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks** *IEEE ICOIN, 2026* [37]	Kubernetes; compared with HPA and KEDA	LSTM forecasts future load; a DQN chooses proactive scaling actions	Open-source hybrid forecast-plus-DRL pipeline	Separating predictor and RL benefit; benchmark breadth
29	**ADASCALE: An Adaptive Scaling and Placement Framework for Microservices Under Dynamics** *arXiv preprint, 2026* [38]	Cloud-edge Kubernetes; DeathStarBench Social Network	MAPE loop with traces and service-mesh metrics; SLO-aware replica targets plus latency-aware placement	Treats scaling and placement as one coupled problem	Preprint; edge-specific assumptions; attribution of gains

2.1 SUMMARY OF THE LITERATURE SURVEY

This section condenses the surveyed work under four heads: the problem faced, the traditional approaches used, the solutions proposed in the literature, and the key takeaways.

Problem Faced

- **Variable and bursty demand:** microservice workloads change by the minute, and spikes are short but costly. A controller sized for the average misses the peaks; one sized for the peak wastes resources most of the time [10], [16].
- **Actuation delay:** metric collection, decision, scheduling, image pull and readiness checks mean that new capacity arrives after the load has already risen [13], [22].
- **Inter-service dependencies:** scaling one service can expose or shift a bottleneck in another, so isolated per-service decisions are often wrong [12], [24], [31].
- **Conflicting objectives:** latency, cost, energy and scaling stability pull in different directions, and the right balance depends on the business [32], [33].

Traditional Approaches Used

- **Threshold-based reactive scaling** (Kubernetes HPA, cloud provider rules) adjusts replicas when a metric crosses a target. It is simple and transparent but reacts late and needs manual tuning [11], [5].
- **Static provisioning and scheduled scaling** fix capacity or follow a calendar; they cannot handle unexpected bursts.
- **Queueing and control-theory models** estimate the needed capacity from arrival and service rates or from feedback error. They are interpretable but assume stable parameters and must be recalibrated when the workload changes [15].

Solutions Proposed in the Literature

- **Predictive ML** (regression, Random Forest, Bi-LSTM, SCINet, ARIMA) forecasts demand so that scaling can begin before the load arrives [12], [13], [21], [22].
- **Reinforcement learning** (DQN, A2C, PPO, TD3, meta-RL) learns the scaling policy itself from reward feedback and can balance several objectives over time [18], [19], [20], [25], [31].
- **Dependency-aware and graph models** (GNNs over traces, architecture-level global scaling) coordinate decisions across the service graph [24], [29].
- **Hybrid and guarded controllers** combine forecasts with reactive feedback, rollback and fallback rules, or split the problem into a learned global layer and simple local controllers [23], [26], [30], [35].
- **Multi-layer, energy-aware and placement-aware scaling** widens the decision from “how many Pods?” to VMs, energy and where replicas run [32], [33], [38].

Table 2.2: Comparative synthesis of autoscaling approach families

Approach	Information used	Main strength	Main risk	Representative work
Reactive threshold (HPA)	Current CPU / memory or custom metric	Simple, mature, easy to explain	Late reaction; single metric; no dependencies	Kubernetes HPA [5], HPAQT [34]
Predictive ML	Metric history and forecasts	Acts before demand arrives	Forecast error, drift and missed bursts	[12], [13], [16], [21], [22], [17]
RL / Deep RL	State, action and reward over time	Learns delayed trade-offs and multi-objective policies	Training cost, unsafe exploration, reward design	[18], [19], [20], [25], [31], [37]
Graph / dependency-aware	Call graph, traces, service-mesh data	Finds the real bottleneck; coordinated scaling	Graph construction, incomplete traces, overhead	[24], [27], [29], [28], [38]
Hybrid and guarded	Forecast plus feedback plus rules	Combines early action with safety	More components to tune and validate	[23], [26], [30], [35]
Multi-layer / energy-aware	Pods, VMs, energy model	Lower infrastructure cost and energy	Cross-layer timing and migration cost	[32], [33]

Key Takeaways

- Learning-based autoscalers consistently beat the default HPA in the authors' own experiments: for example, up to 20% better response time with RL-tuned thresholds [11], up to 19% CPU saving with a GNN [24], and up to 50% lower latency with about 30% lower cost when an RL agent observes all microservices together [31].
- Simple models are often enough. Random Forest [21], ARIMA and even a naive "last value" forecast [31] perform close to deep models at a fraction of the cost; LSTMs help mainly for sudden changes.
- Training RL agents on a real cluster is expensive. In Gwydion the cluster mode was on average about one hundred times more costly than the simulation mode, yet agents trained in simulation performed similarly when tested, which supports offline training followed by real validation [31].
- Production evidence exists. Google's Autopilot halved the slack between reserved and used resources compared with manual settings [14], showing that automatic, data-driven sizing pays off at fleet scale.
- Safety is becoming a first-class design goal. Production work adds bootstrapping, rollback and fallback rather than relying on the learned policy alone [23], [35].

Overall Observations

- **Common problem:** reactive, per-service, CPU-driven scaling cannot keep latency, cost and stability in balance for dependent microservices under changing load.
- **Common solution:** add prediction to act earlier, dependency information to act in the right place, learning to balance objectives, and guardrails to act safely.
- **Common limitation:** each added layer of intelligence adds training cost, complexity and risk, and few studies combine prediction, dependency awareness and safety in one controller; this motivates Chapter 4.

3. MOTIVATION, PURPOSE, SCOPE AND OBJECTIVE OF SEMINAR

3.1 Motivation and Purpose

- Cloud-native applications are now the default way to build large online services, and almost all of them run on Kubernetes. Every one of these deployments has to decide, continuously, how many replicas each service should run. Getting this wrong is costly in both directions: idle replicas waste money and energy, and missing replicas break SLOs and drive users away.
- The industry default, the Horizontal Pod Autoscaler, is reliable but structurally late and blind to dependencies. The research literature offers many "intelligent" alternatives, but they are scattered across prediction, reinforcement learning, graph learning and control theory, each solving part of the problem. Fig. 3.1 summarises the practical challenges that make the problem hard.

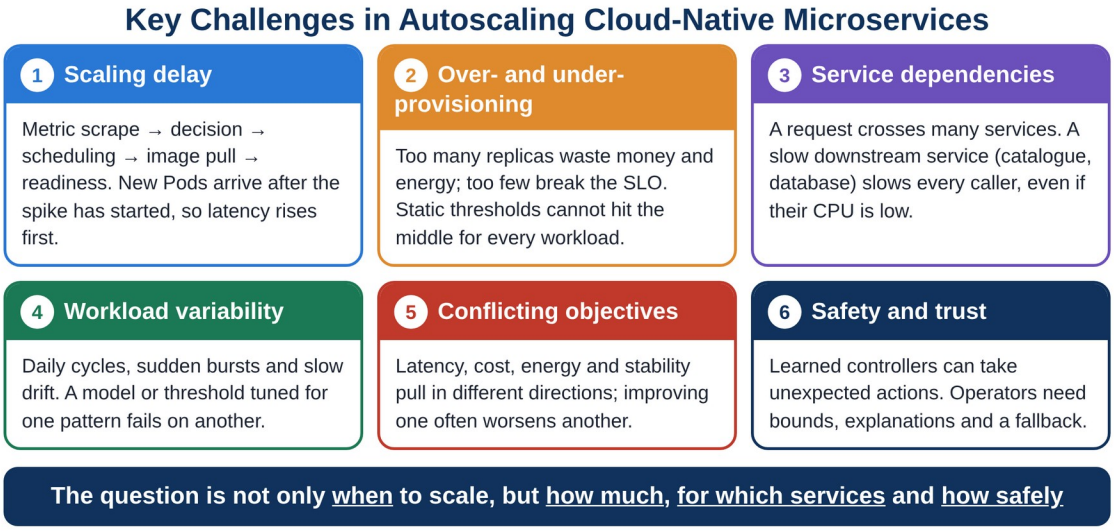


Fig. 3.1: Key challenges in autoscaling cloud-native microservices

- The **purpose** of this seminar is to understand these approaches from first principles, compare them on a common basis, identify what remains unsolved, and propose a defensible framework and evaluation plan that combines prediction, dependency awareness and safety. The work is analytical and literature-based; it does not claim a new, experimentally validated autoscaler.

3.2 Scope

- Covers **horizontal autoscaling of microservices on Kubernetes** as the main focus, with vertical, node-level, multi-layer and energy-aware scaling discussed where they affect the design.
- Surveys **twenty-nine studies published between 2019 and 2026** together with recent surveys, comparing their environments, methods, results and limitations.

- Explains the underlying concepts: the HPA control loop, MAPE-K, time-series forecasting, Markov decision processes, Q-learning, policy-gradient methods (A2C, PPO), dependency graphs and safety constraints.
- Proposes a conceptual framework, algorithm and evaluation plan. **No new experiments are carried out**; all quantitative results belong to the original authors.
- Out of scope: serverless function scaling, general load balancing, VM scheduling algorithms and claims of production performance without supporting experiments.

3.3 Objectives

- To understand the autoscaling problem in cloud-native microservices, including demand variability, actuation delay and service dependencies.
- To analyse the working and limitations of the **Kubernetes HPA** as the baseline controller.
- To study how **workload prediction, reinforcement learning and graph-based dependency models** improve scaling decisions.
- To compare existing intelligent autoscalers and identify their **research gaps** in safety, coordination, multi-objective optimisation and evaluation.
- To propose a **predictive, dependency-aware and constrained autoscaling framework** with a clear algorithm and an evaluation plan against a tuned HPA baseline.

3.4 Challenges and Possible Safeguards

Table 3.1 lists the main practical challenges met when intelligent autoscaling is used on a real cluster, and the safeguards that the proposed design uses against each of them.

Table 3.1: Practical challenges in intelligent autoscaling and possible safeguards

Challenge	Why it matters	Possible safeguard
Traffic bursts	Capacity can arrive after the spike has already harmed latency.	Short-horizon forecasting, headroom margin and burst tests.
Forecast error	Over-estimation wastes resources; under-estimation risks the SLO.	Conservative margins, reactive HPA correction and fallback.
Service dependencies	Scaling one service can expose a downstream bottleneck.	Call-graph features and coordinated scaling of dependent services.
Oscillation (thrashing)	Frequent scale-up / scale-down adds overhead and instability.	Smoothing, tolerance band, cooldown and stabilisation window.
Unsafe exploration	A poor learned action can affect real users.	Train offline in simulation, bound actions, shadow mode and rollback.
Workload drift	A model trained on last month may fail next month.	Drift detection, periodic re-evaluation and safe retraining.
Inconsistent evaluation	Different baselines and cost models make results incomparable.	Fixed protocol, repeated trials, full configuration reporting.

4. METHODOLOGY AND ALGORITHMS

This chapter presents the methodology followed in this seminar and the algorithms behind intelligent autoscaling. It starts from the Kubernetes HPA baseline and then adds, one layer at a time, the components that the literature shows to be useful: **workload prediction**, **dependency analysis**, a **multi-objective learned policy** and a **safety / constraint layer**. Together they form the proposed **Predictive, Dependency-Aware and Constrained Autoscaling Framework** shown in Fig. 4.1, which was first presented in Seminar Review I. The chapter ends with the complete algorithm, the staged methodology and the evaluation plan.

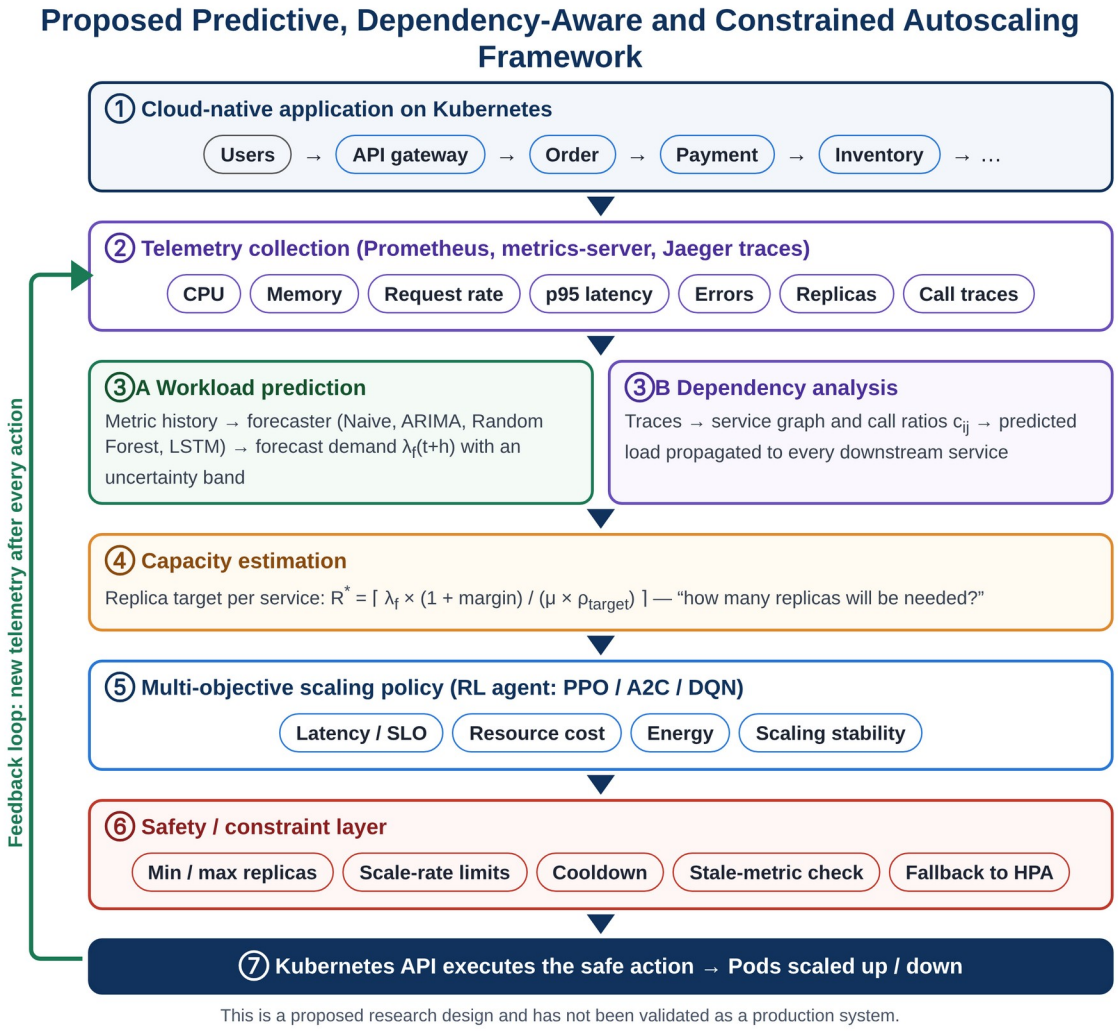


Fig. 4.1: Proposed predictive, dependency-aware and constrained autoscaling framework

4.1 Problem Formulation and Overall Approach

- **System:** an application *a* made of a set of microservices (Kubernetes Deployments) $j = 1 \dots M$. Each service runs R_j replicas, each replica can serve about μ_j requests per second at the target utilisation, and the application receives an external request rate $\lambda_0(t)$.

- **Decision:** at each control interval, choose the replica count $R_j(t)$ of every service (horizontal scaling). Vertical and node scaling are treated as constraints, not decisions.
- **Objectives:** keep the end-to-end p95 latency within the SLO, minimise the number of replica-hours (cost), and avoid unnecessary scaling actions (churn).
- **Constraints:** $R_{min} \leq R_j \leq R_{max}$, limited scale-up and scale-down per interval, cooldown after scale-down, and the cluster's total CPU and memory.
- A useful first-principles check is the **utilisation** of a service, the ratio of arriving work to available capacity. If ρ approaches 1, queues and latency grow sharply, which is why autoscalers target a utilisation well below 100% (for example 60–75%).

$$\rho = \frac{\lambda}{N \cdot \mu} < 1 \tag{4.1}$$

Here λ is the request arrival rate, N the number of replicas and μ the service rate of one replica.

4.2 Baseline: Kubernetes Horizontal Pod Autoscaler

- The HPA is a **control loop** that runs periodically (by default every 15 seconds). It reads resource metrics from the metrics-server or custom metrics from Prometheus, compares them with a target, and updates the replica count of the Deployment [5], [46]. The loop follows the classic **MAPE-K** pattern of autonomic computing: Monitor, Analyse, Plan and Execute over shared Knowledge [47] (Fig. 4.2).

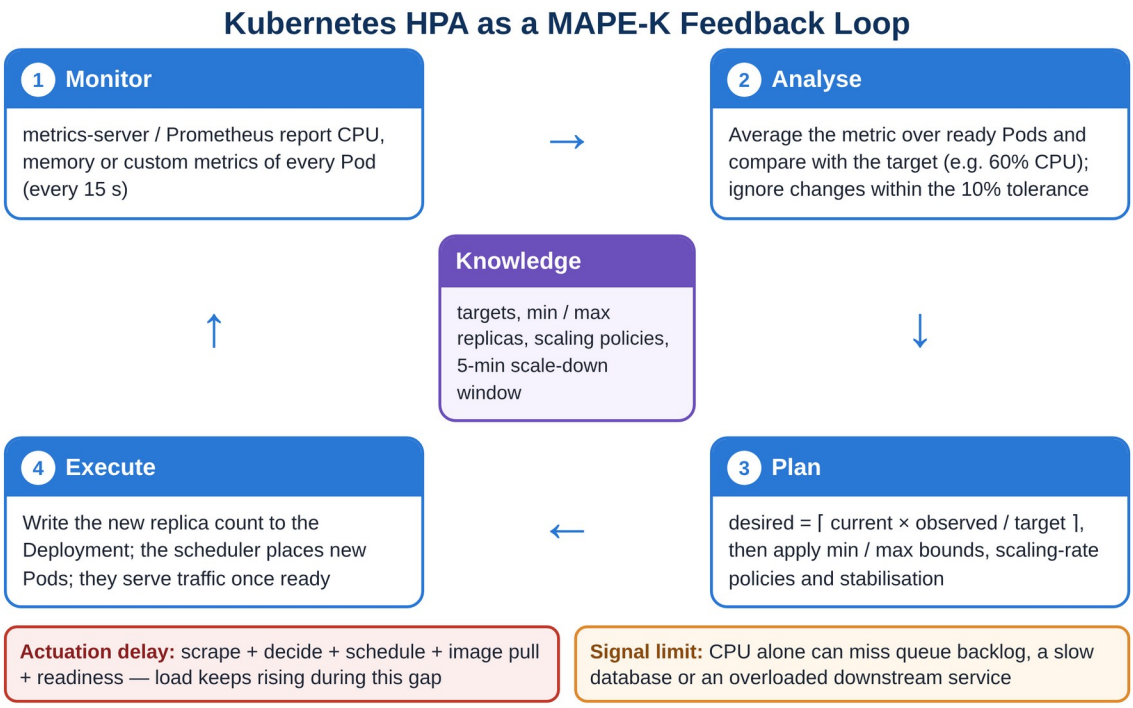


Fig. 4.2: Reactive autoscaling as a MAPE-K feedback loop in the Kubernetes HPA

- The desired number of replicas is computed from the ratio of the observed metric to the target value:

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{desiredMetricValue} \right\rceil \tag{4.2}$$

- **Default behaviour:** when several metrics are configured, the HPA computes a replica count for each metric and uses the largest. By default it may at most double the replicas, or add four Pods, every 15 seconds when scaling up, and when scaling down it uses the highest recommendation of the previous five minutes, so capacity is released slowly [5].
- **Worked example:** a service runs 4 replicas at 80% average CPU with a 50% target. The ratio is 80 / 50 = 1.6, so the HPA requests $\lceil 4 \times 1.6 \rceil = 7$ replicas. A tolerance band (10% by default) avoids acting on tiny differences, and a scale-down stabilisation window (300 seconds by default) prevents rapid flapping [5].
- **Related Kubernetes mechanisms:** the Vertical Pod Autoscaler recommends or applies CPU and memory requests [6]; node autoscalers add or remove worker nodes [8]; and KEDA scales on external events such as queue length [9].
- **Limitations of the baseline:** (i) it is **reactive**, so it always lags behind a rising load; (ii) it scales each Deployment **independently** without knowing the call graph; (iii) it relies on **resource metrics** that may not reflect latency; and (iv) its thresholds must be **tuned by hand**. Fig. 4.3 illustrates the first limitation with a simple synthetic example.

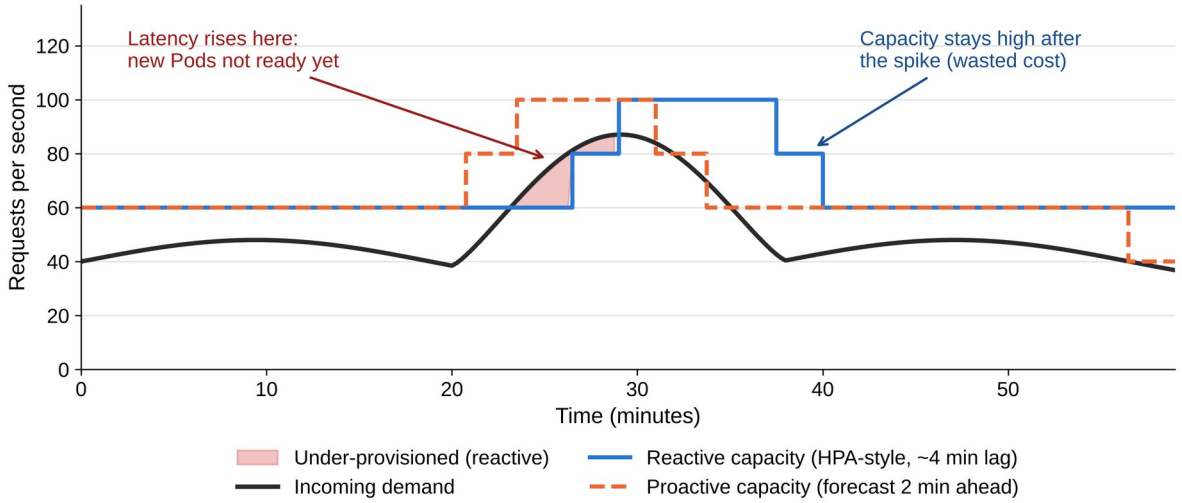


Fig. 4.3: Illustrative example (synthetic data): reactive capacity lags a demand spike, while forecast-driven capacity rises in time

4.3 Workload Prediction Module

- The prediction module estimates the demand of the next interval so that capacity can be added **before** it is needed. Inputs are time-stamped metrics: request rate, CPU and memory, with lagged values, rolling means and time-of-day features. Data must be split **chronologically** into training, validation and test periods to avoid leaking future information.

- **Candidate models:** a **Naive** forecast (next value equals the last value) as a benchmark; **ARIMA** for short-term linear patterns; **Random Forest** for non-linear relations with little tuning [21]; and **LSTM / Bi-LSTM** networks for long-range and sudden patterns [13]. In Gwydion, ARIMA and the Naive forecast gave the lowest errors for most services when samples were collected every few seconds, while LSTM was most useful for a service with sudden CPU changes but needed more computation [31].
- **Forecast accuracy** is measured with the Mean Absolute Error and the Root Mean Square Error, where y_t is the actual and $\hat{y}_t$ the predicted value; RMSE penalises large misses more strongly:

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2} \tag{4.3}$$

- **Key point:** a low average error does not guarantee good scaling. A model can be accurate on most intervals yet miss the one burst that matters [16]. Forecast quality and end-to-end scaling quality must therefore be evaluated separately, and the forecast should feed a decision layer rather than act directly.

4.4 Dependency Analysis Module

- Microservice applications form a **service dependency graph** G = (V, E), where an edge $i \rightarrow j$ means that service *i* calls service *j*. Fig. 4.4 shows the graph of **Online Boutique**, the eleven-service e-commerce benchmark used by gym-hpa and Gwydion [20], [31]. Checkout is a fan-out hub that calls six services, and Product catalog is shared by three callers, so a surge at the Frontend reaches it along three paths.

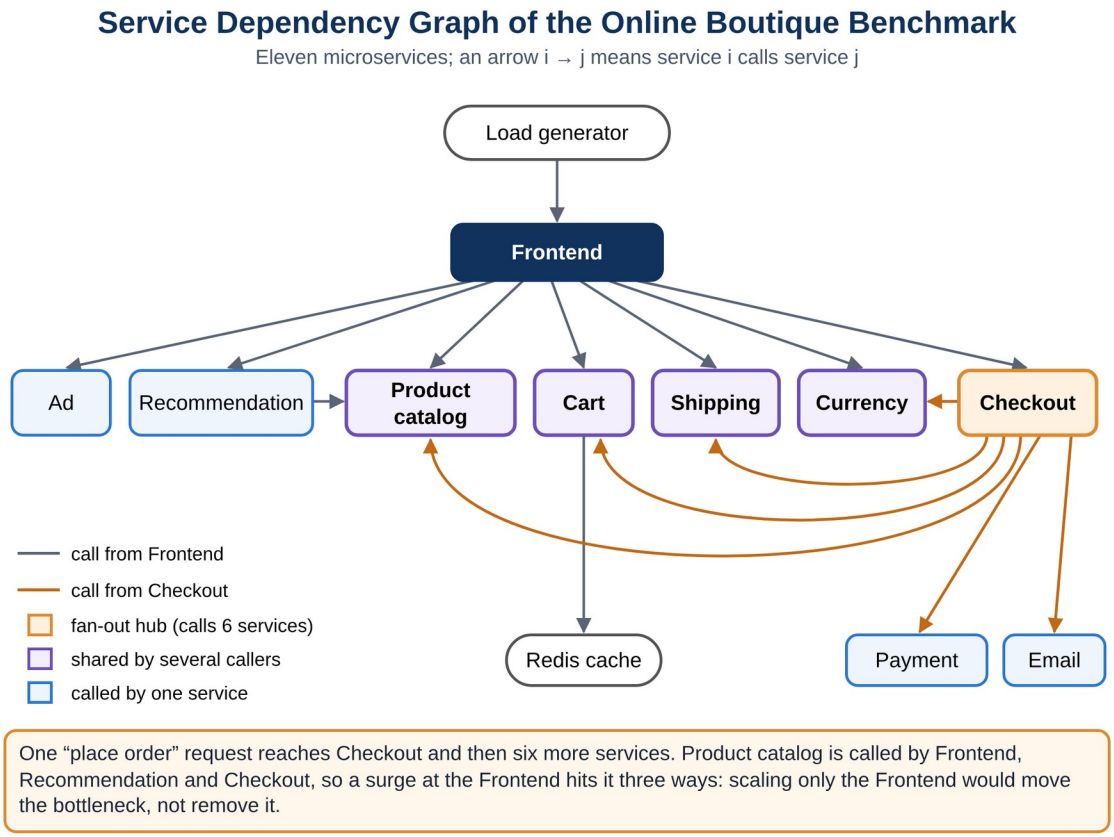


Fig. 4.4: Service dependency graph of the Online Boutique benchmark (eleven microservices)

- The graph is built from distributed traces (Jaeger), service-mesh metrics (Istio) or static configuration. Each edge carries a **call ratio** c_{ij}, the average number of calls service *i* makes to service *j* per request it handles. The expected load on service *j* is then the sum of the load forwarded by its callers, or equivalently the front-end rate λ_0 times the visit ratio v_j:

$$\lambda_j = \lambda_0 \, v_j = \sum_{i \in \text{callers}(j)} \lambda_i \, c_{ij} \tag{4.4}$$

- This lets the controller scale downstream services **at the same time** as the front end, instead of waiting for each service’s own CPU to rise. Gwydion’s data confirm the effect: the number of follower Pods in Redis Cluster correlates strongly with the leader’s CPU, and the frontend CPU in Online Boutique correlates with the product-catalogue CPU [31]. GRAF and DeepScaler use graph neural networks for the same purpose when relations are non-linear [24], [27].

4.5 Capacity Estimation

- The predicted load of each service is converted into a replica target using the capacity of one replica, a target utilisation and a safety margin *m* (for example 10–20%) for forecast error:

$$R_j^*(t) = \left\lceil \frac{\hat{\lambda}_j(t+h)\,(1+m)}{\mu_j\,\rho_{\text{target}}} \right\rceil \tag{4.5}$$

- Here h is the forecast horizon, chosen to be at least the measured Pod start-up time so that new replicas are ready when the load arrives. The per-replica capacity μ_j is measured by load-testing each service once, or learned from history. The result R^*_j is a **recommendation**; the policy and safety layers decide whether and how to apply it.
- **Uncertainty-aware version:** if the forecaster also returns a standard deviation σ_j, the controller can provision for a chosen quantile instead of the mean. With z = 1.65 the target covers about 95% of outcomes under a normal assumption; the margin then grows when the forecast is unreliable and shrinks when it is accurate, which is the idea behind MagicScaler [17]:

$$R_j^*(t) = \left\lceil \frac{\hat{\lambda}_j(t+h) \;+\; z\,\sigma_j(t+h)}{\mu_j\,\rho_{\text{target}}} \right\rceil \tag{4.6}$$

- **Worked example (illustrative values):** the forecaster predicts $\hat{\lambda}_0$ = 400 requests/s at the Frontend for the next interval. With a 15% margin (m = 0.15), a 70% target utilisation and per-replica capacities measured by load testing, Eq. 4.4 and Eq. 4.5 give the replica targets in Table 4.1. All six services are scaled **together, before the load arrives**; a per-service HPA would scale Product catalog and Currency only after their own CPU had already risen.

Table 4.1: Worked example of dependency-aware capacity estimation (assumed values)

Service	**Visit ratio v_j**	**Load $\hat{\lambda}_j$ (req/s)**	**Capacity μ_j (req/s per replica)**	**Raw value before rounding**	**Replicas R^*_j**
Frontend	1	400	120	5.48	**6**
Product catalog	1.6	640	250	4.21	**5**
Recommendation	0.5	200	80	4.11	**5**
Currency	1.2	480	400	1.97	**2**
Cart	0.6	240	200	1.97	**2**
Checkout	0.05	20	40	0.82	**1**

4.6 Multi-Objective Scaling Policy using Reinforcement Learning

- A fixed formula cannot weigh latency against cost and stability when effects are delayed and coupled. **Reinforcement learning (RL)** treats autoscaling as a sequence of decisions: an **agent** observes the **state** of the cluster, takes a scaling **action**, and receives a **reward** that reflects how good the result was [48] (Fig. 4.5).

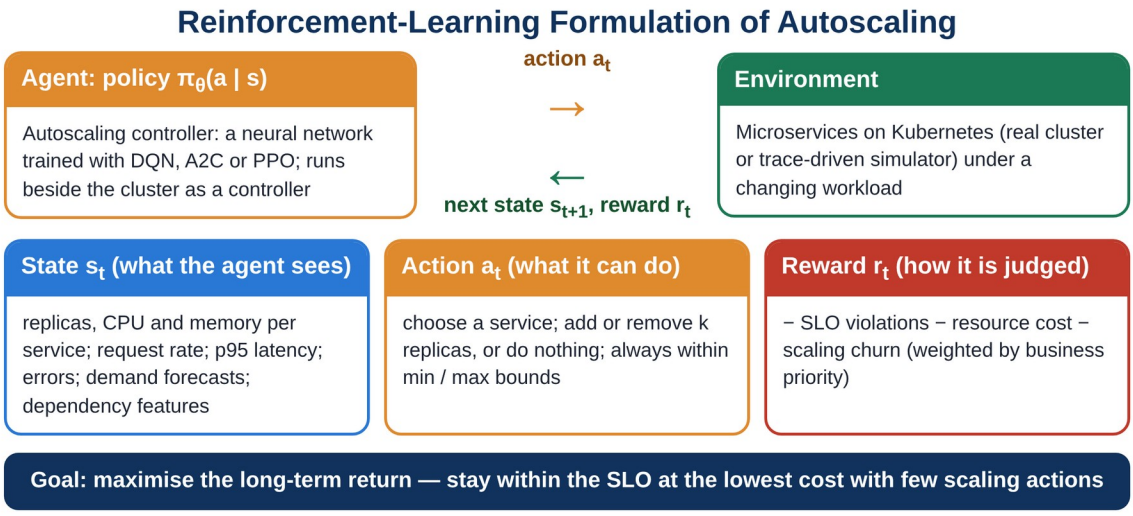


Fig. 4.5: Reinforcement-learning formulation of microservice autoscaling

4.6.1 Markov Decision Process

- **State s_t:** for every service, the replicas, CPU and memory usage, request rate, p95 latency and error rate, plus the forecasts and replica recommendations from Sections 4.3–4.5. Gwydion also adds a counter of consecutive "do nothing" actions so that the agent does not get stuck in an idle policy, and normalises all values with min–max scaling [31].
- **Action a_t:** a multi-discrete choice of *which service* to act on and *what to do*: do nothing, or add or remove k replicas, where k is bounded by the replica limits. With limits of 1 and 8 replicas this gives fifteen scaling choices for each service [31].
- **Return:** the agent maximises the discounted sum of future rewards, where the discount factor γ sets how much future outcomes matter:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \qquad 0 \le \gamma < 1 \tag{4.7}$$

4.6.2 Value-Based Learning: Q-Learning and DQN

- The **action-value** Q(s, a) is the expected return of taking action *a* in state *s* and acting well afterwards. The optimal values satisfy the **Bellman equation**:

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \right] \tag{4.8}$$

- A Q-table is impossible for continuous cluster metrics, so a **Deep Q-Network (DQN)** approximates Q with a neural network, trained from a replay buffer with a periodically updated target network [49]. DQN suits small, discrete action sets and is used in recent hybrid autoscalers such as NimbusGuard [37].

4.6.3 Policy-Gradient Learning: A2C and PPO

- **Actor-critic** methods learn a policy $\pi_\theta(a \mid s)$ (the actor) together with a value estimate (the critic). **A2C** (Advantage Actor-Critic) updates the actor in the direction of actions whose

advantage $\hat{A}_t$ is positive [50]. **PPO** (Proximal Policy Optimisation) limits how far each update can move the policy by clipping the probability ratio $k_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta\text{-old}}(a_t \mid s_t)$, which makes training more stable [51]:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(k_t(\theta)\hat{A}_t,\ \mathrm{clip}(k_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right] \tag{4.9}$$

- In Gwydion, A2C gave the most balanced results across cost-aware and latency-aware goals, while PPO and recurrent PPO reached low latency by deploying many more Pods [31]. This shows that the reward, not only the algorithm, decides the behaviour.

4.6.4 Reward Design

- The reward is the most important design choice. The proposed policy uses a weighted penalty on SLO violations, resource cost (replica count or CPU-seconds) and churn (size of the scaling change):

$$r_t = -\,w_1\,\mathrm{SLO}_{\mathrm{viol}}(t)\ -\ w_2\,\mathrm{Cost}(t)\ -\ w_3\,|\Delta R(t)| \tag{4.10}$$

- The weights w_1, w_2 and w_3 express business priorities. If w_1 is too small the agent saves money but violates the SLO; if it is too large the agent over-provisions. Gwydion's two opposite rewards illustrate this: a **cost-aware** reward that gives +1 when the replica count matches the demand-based target, and a **latency-aware** reward equal to the negative application latency, with an extra penalty when latency exceeds its threshold [31].

4.6.5 Training Environment

- Training directly on a production cluster is slow and risky. Gwydion provides an OpenAI Gym environment with two modes (Fig. 4.6): a **cluster mode** that acts on a real Kubernetes cluster through the Kubernetes and Prometheus APIs, and a **simulation mode** that replays datasets collected from the cluster. Training for 2,500 episodes took a few minutes to about three hours in simulation, but two to four days on the real cluster [31].

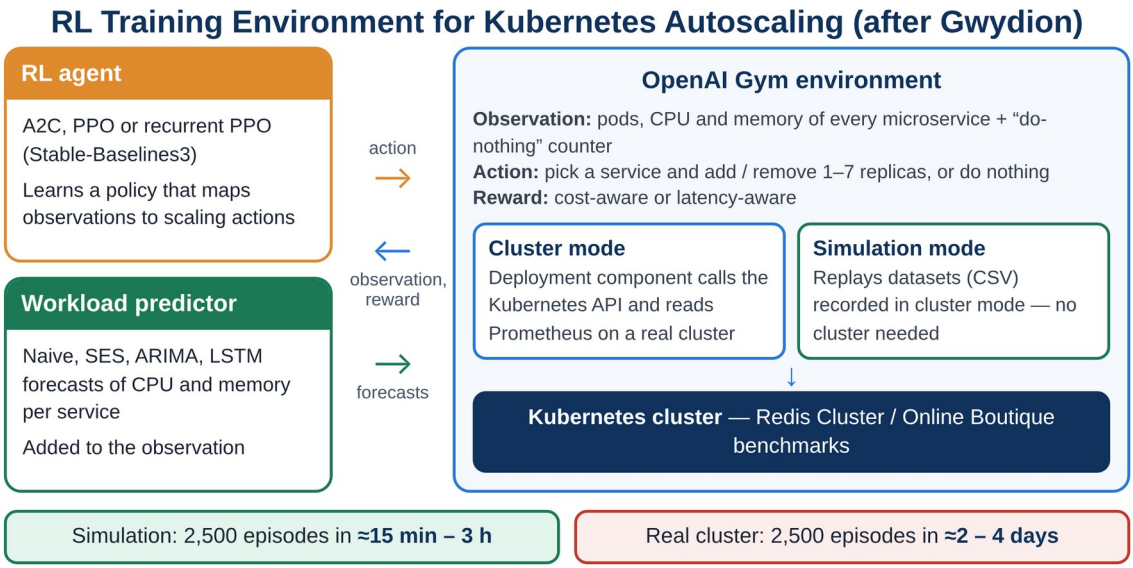


Fig. 4.6: RL training environment with cluster and simulation modes, redrawn after the Gwydion framework [31]

- The proposed methodology therefore **trains offline** on recorded traces or in simulation, validates in a test cluster, and runs the policy in **shadow mode** (recommend without acting) before it is allowed to control real replicas, following the production practice of AWARE [23].

4.7 Safety and Constraint Layer

- Every action, whether from the forecaster or the RL policy, passes through a deterministic safety layer before execution. The safe replica target is clipped to the allowed bounds and to the maximum change per interval:

$$R_{\text{safe}}(t) = \text{clip}\left(R^{*}(t),\ \max\left(R_{\min},\ R(t) - \Delta_{\downarrow}\right),\ \min\left(R_{\max},\ R(t) + \Delta_{\uparrow}\right)\right) \tag{4.11}$$

- **Guardrails:** minimum and maximum replicas; scale-up and scale-down rate limits; a **cooldown** after each scale-down; readiness checks before traffic is sent to new Pods; and protection against acting on missing or stale metrics.
- **Fallback and rollback:** if the model fails, telemetry is missing, or SLO violations persist for several intervals, control returns to the standard HPA rule (Eq. 4.2). The safety-net study shows that such a rollback guardrail is a practical way to use forecasts without trusting them blindly [35].
- **Auditability:** each decision, its inputs and the reason for any override are logged, so that operators can explain and reproduce the controller's behaviour.

4.8 Complete Algorithm

The steps above are combined into one closed-loop controller, summarised in Algorithm 1.

Algorithm 1: Predictive, dependency-aware and constrained autoscaling loop

Line	Operation
1	Initialise replica bounds R_{min}, R_{max}, scale limits $\Delta\uparrow$, $\Delta\downarrow$, cooldown τ, forecaster f, policy π_θ and the service graph G.
2	**for** every control interval t **do**
3	Collect telemetry for each service j: CPU, memory, request rate λ_j, p95 latency, errors and replicas R_j.
4	**if** telemetry is missing or stale **then** use the HPA rule (Eq. 4.2) and **continue**.
5	Forecast front-end demand $\hat{\lambda}_0(t+h)$ with f (ARIMA / LSTM / Random Forest).
6	Propagate demand through G to obtain $\hat{\lambda}_j(t+h)$ for every service (Eq. 4.4).
7	Estimate the required replicas R^*_j from predicted load and per-replica capacity (Eq. 4.5).
8	Build state s_t = (telemetry, forecasts, R^*, graph features) and choose action $a_t = \pi_\theta(s_t)$.
9	Apply the safety layer: clip to bounds and scale-rate limits (Eq. 4.11); block scale-down during cooldown.
10	Execute the safe target through the Kubernetes API (patch Deployment replicas).
11	Observe s_{t+1} and compute reward r_t from SLO violations, cost and churn (Eq. 4.10).
12	Store (s_t, a_t, r_t, s_{t+1}); update π_θ offline or in shadow mode with PPO / DQN.
13	**if** SLO violations persist for k intervals **then** roll back to the HPA policy and raise an alert.
14	**end for**

4.9 Proposed Methodology in Stages

- The complete method is organised into nine stages that run from defining the workload to continuous adaptation (Fig. 4.7 and Table 4.2). Each stage produces evidence needed by the next, so an intelligent policy is never evaluated before the workload, baseline and metrics are understood.

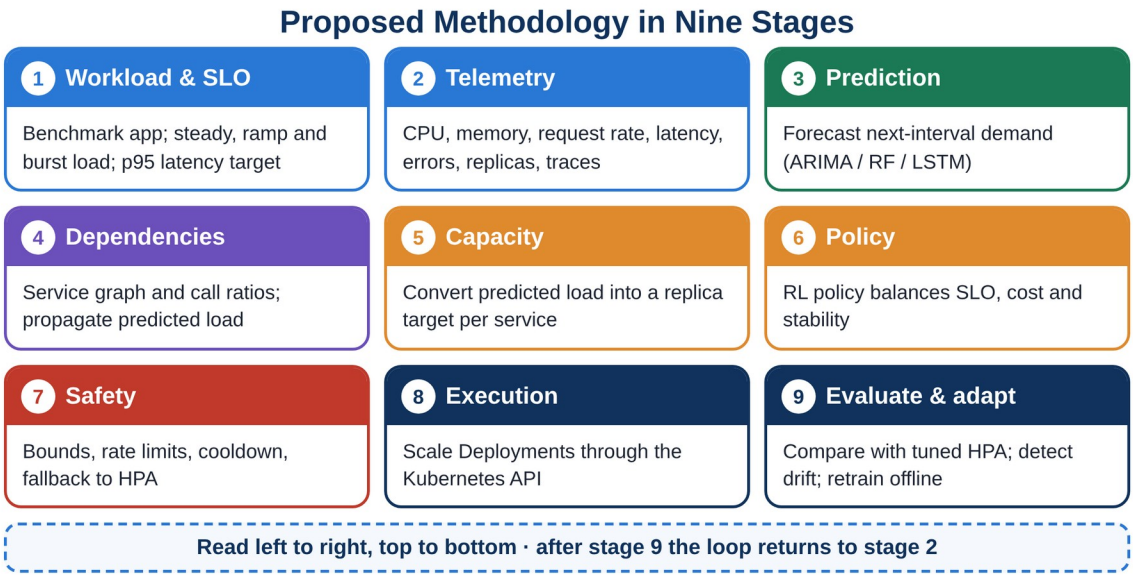


Fig. 4.7: Proposed methodology for predictive, dependency-aware and constrained autoscaling

Table 4.2: Stage-wise description of the proposed methodology

Stage	Name	Activity	Technique Used
1	**Workload and SLO definition**	Deploy a benchmark application and generate steady, ramp, burst and shifting load; fix the p95 latency SLO.	Online Boutique / TeaStore, Locust load generator
2	**Telemetry collection**	Collect CPU, memory, request rate, latency, errors, replicas and traces at a fixed interval.	metrics-server, Prometheus, Jaeger / Istio
3	**Workload prediction**	Forecast next-interval demand for the entry service and evaluate MAE / RMSE.	Naive, ARIMA, Random Forest, LSTM
4	**Dependency analysis**	Build the service graph and call ratios; propagate predicted load to every service.	Trace analysis, Eq. 4.4
5	**Capacity estimation**	Convert predicted load into a replica recommendation per service with a safety margin.	Eq. 4.5, load-tested capacity μ
6	**Multi-objective policy**	Select scaling actions that balance SLO, cost and churn.	PPO / A2C / DQN with Eq. 4.10 reward
7	**Safety / constraint layer**	Clip actions, enforce cooldown and rate limits, fall back to HPA on failure.	Eq. 4.11, rule-based guardrails
8	**Kubernetes execution**	Apply the safe replica targets to the Deployments.	Kubernetes API, Python client
9	**Evaluation and adaptation**	Compare against tuned HPA; detect drift and retrain offline; repeat the loop.	Metrics in Table 4.3; shadow mode

4.10 Evaluation Plan

- **Benchmarks and traces:** TeaStore [52], DeathStarBench [53] and Online Boutique for controlled experiments, and the Alibaba microservice traces [54] for realistic request patterns and call graphs.
- **Baseline:** a *tuned* HPA, not a weak default configuration, so that any improvement is meaningful. Every candidate uses the same workload, resource requests, cluster and measurement window.
- **Metrics and experiments:** Table 4.3 lists the evaluation metrics and their caveats, and Table 4.4 lists five experiments that build on one another, from a fair baseline to safety testing.

Table 4.3: Evaluation metrics for the proposed autoscaler

Metric	What it measures	Caveat
Average latency	Mean response time	Can hide slow tail requests
p95 / p99 latency	Tail response time seen by users	Needs enough requests for stable estimates
SLO violation rate	Fraction of time or requests outside the target	SLO and window must be stated explicitly
Throughput	Completed requests per second	High throughput can coexist with poor latency
Resource cost	Replica-hours, CPU and memory requested	State the cost model; include nodes if scaled
Forecast MAE / RMSE	Prediction error (Eq. 4.3)	Low error may still miss bursts
Scaling churn	Number and size of scaling actions	Frequent actions add overhead and instability
Readiness delay	Time until new Pods serve traffic	Decides the minimum useful forecast horizon

Table 4.4: Planned experiments

Experiment	Comparison	Question investigated
E1: Baseline	Default HPA vs tuned HPA	What does a fair, well-configured reactive baseline achieve?
E2: Forecast	Tuned HPA vs forecast-then-scale	Does early action reduce SLO violations enough to justify forecast errors?
E3: Workload shift	Train on one period, test on another	How much does performance degrade under drift and new bursts?
E4: Dependency	Independent vs graph-aware scaling	Does coordinated scaling improve end-to-end latency when the bottleneck moves?
E5: Safety	Policy with vs without guardrails	How many unsafe actions, oscillations and rollbacks occur?

4.11 Improvements of the Proposed Framework over Kubernetes HPA

Table 4.5 compares the proposed framework with the Kubernetes HPA, component by component.

Table 4.5: Comparison of Kubernetes HPA and the proposed framework

Component	Kubernetes HPA	Proposed framework	Benefit
Timing	Reactive: acts after the metric changes	Proactive: acts on forecast demand, with reactive correction	Capacity is ready before the spike
Scope	Each Deployment scaled independently	Load propagated through the service graph	Bottlenecks are scaled together
Signals	Mainly CPU / memory utilisation	Request rate, latency, errors, forecasts and graph features	Decisions follow user experience
Decision logic	Fixed ratio formula and hand-tuned target	Learned multi-objective policy (PPO / A2C / DQN)	Explicit SLO–cost–churn trade-off
Safety	Bounds and stabilisation window	Bounds, rate limits, cooldown, shadow mode, rollback to HPA	Learning without risking users
Adaptation	Manual retuning	Drift detection and offline retraining	Keeps up with changing workloads

- **Expected outcome:** based on the results reported in the surveyed studies, the combination is expected to lower SLO violations during bursts and to reduce over-provisioning compared with a tuned HPA. These are hypotheses to be tested with the plan in Section 4.10, not measured results.

5. DISCUSSION AND CONCLUSION

5.1 The Core Problem

- Cloud-native microservice applications face changing and bursty demand while having to keep **latency within the SLO, cost low and the system stable**. The default Kubernetes HPA is simple and dependable but reacts late, relies on resource metrics and scales each service in isolation, so it can both miss spikes and over-provision after them.

5.2 Discussion

- The survey shows that no single algorithm is best in every setting. **Prediction** is most valuable when patterns are learnable and Pod start-up is slow; **reinforcement learning** is most valuable when actions have delayed, coupled effects and several objectives must be balanced; **graph models** matter when dependencies decide end-to-end latency; and **guardrails** decide whether any of these can be trusted in production.
- Realistic evaluation matters. Production traces contain thousands of services with changing, long-tailed dependencies [39], [40], far from the ten-service benchmarks used in most studies, so results on small benchmarks should be treated as early evidence.
- Simple methods deserve respect. Random Forest, ARIMA and even naive forecasts often perform close to deep models [21], [31], and a well-tuned HPA remains the right baseline. Added complexity should be justified by measured gains.
- A useful way to compare autoscalers is to ask six questions: what signals are observed, how they are represented, what is predicted or learned, which resource layer is changed, which objective is optimised, and how realistic and reproducible the evaluation is.

5.3 Expected Impact and Contributions

- A structured comparison of **twenty-nine studies** that maps the field by trigger, target layer, decision method, scope, objective and evidence.
- A clear identification of **research gaps**: exploration cost, the dependency blind spot, multi-dimensional trade-offs, unguarded prediction and weak evaluation standards.
- A **predictive, dependency-aware and constrained framework** with an explicit algorithm, equations and a nine-stage methodology that can be implemented incrementally.
- An **evaluation plan** with a tuned HPA baseline, defined metrics and five progressive experiments.

5.4 Limitations

- This report is a literature study and proposal; the framework has **not been implemented or validated** in this seminar.
- Reported numbers come from different environments, workloads and baselines and are **not directly comparable**. Several 2024–2026 works are recent or preprints and should be re-checked for final publication status.

- Graph-aware scaling assumes reasonably complete traces, and RL-based control still needs careful reward design and safe training.

5.5 Future Work

- Implement the framework on a small Kubernetes cluster with Online Boutique, starting with a tuned HPA baseline and a simple forecaster before adding RL.
- Represent **forecast uncertainty** (prediction intervals) in the decision so that the safety margin adapts to how reliable the forecast is.
- Extend the policy to **joint horizontal, vertical and node scaling**, and to energy- and carbon-aware objectives.
- Study **safe online learning** and drift detection so that the policy can keep improving without harming users.
- Contribute to **standard, reproducible evaluation** by publishing workloads, configurations and scripts.

5.6 Conclusion

Intelligent autoscaling aims to match the capacity of cloud-native applications to changing demand while protecting user experience and avoiding waste. The Kubernetes HPA provides a reliable reactive baseline, and the literature shows clear progress beyond it: forecasting lets capacity arrive on time, reinforcement learning balances latency, cost and stability, dependency graphs put capacity where the bottleneck really is, and guardrails make learned control safe enough for production. Each of these, however, brings its own costs in training, complexity and risk, and few studies combine them in a single evaluated controller. The most defensible direction is therefore an end-to-end, closed-loop controller that **predicts demand, understands service dependencies, chooses actions with a multi-objective policy and enforces safety constraints with a fallback to HPA**. The framework proposed in this report follows that direction and is ready to be built and tested incrementally against a tuned baseline, so that any claimed improvement is backed by controlled experiments.

REFERENCES

[1] Cloud Native Computing Foundation, “The CNCF Annual Cloud Native Survey,” CNCF Research Report, Jan. 2026. [Online]. Available: https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/

[2] Flexera, “2025 State of the Cloud Report,” Mar. 2025. [Online]. Available: https://www.flexera.com/about-us/press-center/new-flexera-report-finds-84-percent-of-organizations-struggle-to-manage-cloud-spend

[3] Datadog, “State of Cloud Costs 2024,” cited in “Datadog announces Kubernetes Autoscaling to help teams intelligently optimize resources and cloud costs,” press release, Jun. 26, 2024. [Online]. Available: https://www.datadoghq.com/about/latest-news/press-releases/datadog-announces-kubernetes-autoscaling-to-help-teams-intelligently-optimize-resources-and-cloud-costs/

[4] International Energy Agency, *Energy and AI*, IEA Special Report, Paris, Apr. 2025. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[5] Kubernetes Documentation, “Horizontal Pod Autoscaling.” [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[6] Kubernetes Documentation, “Vertical Pod Autoscaling.” [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/

[7] Kubernetes Blog, “Kubernetes 1.35: In-place Pod resize graduates to stable,” Dec. 19, 2025. [Online]. Available: https://kubernetes.io/blog/2025/12/19/kubernetes-v1-35-in-place-pod-resize-ga/

[8] Kubernetes Documentation, “Node Autoscaling.” [Online]. Available: https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/

[9] KEDA, “Kubernetes Event-driven Autoscaling.” [Online]. Available: https://keda.sh/

[10] M. Abdullah, W. Iqbal, A. Mahmood, F. Bukhari, and A. Erradi, “Learning predictive autoscaling policies for cloud-hosted microservices using trace-driven modeling,” in *Proc. IEEE Int. Conf. Cloud Computing Technology and Science (CloudCom)*, 2019, pp. 119–126, doi: 10.1109/CloudCom.2019.00028.

[11] A. Abdel Khaleq and I. Ra, “Intelligent autoscaling of microservices in the cloud for real-time applications,” *IEEE Access*, vol. 9, pp. 35464–35476, 2021, doi: 10.1109/ACCESS.2021.3061890.

[12] A. Goli, N. Mahmoudi, H. Khazaei, and O. Ardakanian, “A holistic machine learning-based autoscaling approach for microservice applications,” in *Proc. 11th Int. Conf. Cloud Computing and Services Science (CLOSER)*, 2021, pp. 190–198.

[13] N.-M. Dang-Quang and M. Yoo, “Deep learning-based autoscaling using bidirectional long short-term memory for Kubernetes,” *Applied Sciences*, vol. 11, no. 9, Art. no. 3835, 2021, doi: 10.3390/app11093835.

[14] K. Rzadca *et al.*, “Autopilot: Workload autoscaling at Google,” in *Proc. 15th European Conf. Computer Systems (EuroSys)*, 2020, doi: 10.1145/3342195.3387524.

[15] A. F. Baarzi and G. Kesidis, “SHOWAR: Right-sizing and efficient scheduling of microservices,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2021, pp. 427–441, doi: 10.1145/3472883.3486999.

[16] M. Abdullah, W. Iqbal, F. Bukhari, and A. Erradi, “Burst-aware predictive autoscaling for containerized microservices,” *IEEE Trans. Services Computing*, vol. 15, no. 2, pp. 1059–1072, 2022, doi: 10.1109/TSC.2020.2995937.

[17] Z. Pan *et al.*, “MagicScaler: Uncertainty-aware, predictive autoscaling,” *Proc. VLDB Endowment*, vol. 16, no. 12, pp. 3808–3821, 2023, doi: 10.14778/3611540.3611566.

[18] “DScaler: A horizontal autoscaler of microservice based on deep reinforcement learning,” in *Proc. 23rd Asia-Pacific Network Operations and Management Symp. (APNOMS)*, 2022, doi: 10.23919/APNOMS56106.2022.9919994.

[19] S. Xue *et al.*, “A meta reinforcement learning approach for predictive autoscaling in the cloud,” in *Proc. 28th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 4290–4299, doi: 10.1145/3534678.3539063.

[20] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, “gym-hpa: Efficient auto-scaling via reinforcement learning for complex microservice-based applications in Kubernetes,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2023, doi: 10.1109/NOMS56928.2023.10154298.

[21] L. M. Al Qassem, T. Stouraitis, E. Damiani, and I. M. Elfadel, “Proactive random-forest autoscaler for microservice resource allocation,” *IEEE Access*, vol. 11, pp. 2570–2585, 2023, doi: 10.1109/ACCESS.2023.3234021.

[22] B. Jeong, J. Baek, S. Park, and Y.-S. Jeong, “Proactive resource autoscaling scheme based on SCINet for high-performance cloud computing,” *IEEE Trans. Cloud Computing*, 2023, doi: 10.1109/TCC.2023.3292378.

[23] H. Qiu *et al.*, “AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems,” in *Proc. USENIX Annual Technical Conf. (ATC)*, 2023, pp. 387–402.

[24] J. Park, B. Choi, C. Lee, and D. Han, “GRAF: A graph neural network-based SLO-aware proactive resource autoscaling framework for microservices,” *IEEE/ACM Trans. Networking*, 2024, doi: 10.1109/TNET.2024.3393427.

[25] H. Bai *et al.*, “DRPC: Distributed reinforcement learning approach for scalable resource provisioning in container-based clusters,” *arXiv preprint* arXiv:2407.10169, 2024.

[26] Z. Wang, P. Li, C.-J. M. Liang, F. Wu, and F. Y. Yan, “Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices,” in *Proc. 21st USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2024.

[27] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, “DeepScaler: Holistic autoscaling for microservices based on spatiotemporal GNN with adaptive graph learning,” *arXiv preprint* arXiv:2309.00859, 2023.

[28] M. Dashtbani and L. Tahvildari, “STaleX: A spatiotemporal-aware adaptive auto-scaling framework for microservices,” *arXiv preprint* arXiv:2501.18734, 2025.

[29] L. Bacchiani *et al.*, “Proactive–reactive microservice architecture global scaling,” *J. Systems and Software*, vol. 220, Art. no. 112262, 2025, doi: 10.1016/j.jss.2024.112262.

[30] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, “Towards resource-efficient reactive and proactive auto-scaling for microservice architectures,” *J. Systems and Software*, vol. 225, Art. no. 112390, 2025, doi: 10.1016/j.jss.2025.112390.

[31] J. Santos, E. Reppas, T. Wauters, B. Volckaert, and F. De Turck, “Gwydion: Efficient auto-scaling for complex containerized applications in Kubernetes through reinforcement learning,” *J. Network and Computer Applications*, vol. 234, Art. no. 104067, 2025, doi: 10.1016/j.jnca.2024.104067.

[32] N. Shafi *et al.*, “CEMA: Cost effective multi-layered autoscaling for microservice based applications,” *J. Network and Computer Applications*, 2025, Art. no. 104266, doi: 10.1016/j.jnca.2025.104266.

[33] H. Li *et al.*, “Energy-aware elastic scaling algorithm for microservices in Kubernetes clouds,” *J. Network and Computer Applications*, 2025, Art. no. 104218, doi: 10.1016/j.jnca.2025.104218.

[34] R. Mayerhofer *et al.*, “HPAQT: Adaptive and interpretable high-level SLO-aware autoscaling with reinforcement learning,” in *Proc. IEEE/ACM Int. Conf. Utility and Cloud Computing (UCC)*, 2025, doi: 10.1145/3773274.3774274.

[35] M. Kozhinov and M. Mazzara, “Predictive autoscaling with a safety net for microservices,” in *Proc. Int. Conf. Advanced Information Networking and Applications (AINA)*, 2026, doi: 10.1007/978-3-032-23304-2_22.

[36] C. Duman and S. Eken, “From reactive to predictive: A pattern-aware framework for Kubernetes autoscaling with LLM integration,” *J. Systems and Software*, Art. no. 112861, 2026, doi: 10.1016/j.jss.2026.112861.

[37] C. Wanigasooriya and I. Ekanayake, “NimbusGuard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks,” in *Proc. Int. Conf. Information Networking (ICOIN)*, 2026, doi: 10.1109/ICOIN68469.2026.11480646; also *arXiv preprint* arXiv:2604.11017.

[38] M. Chen *et al.*, “ADASCALE: An adaptive scaling and placement framework for microservices under dynamics,” *arXiv preprint* arXiv:2607.15681, 2026.

[39] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu, “Characterizing microservice dependency and performance: Alibaba trace analysis,” in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2021, doi: 10.1145/3472883.3487003.

[40] G. Winchester, G. Parisis, G. Xu, and L. Berthouze, “Complexity at scale: A quantitative analysis of an Alibaba microservice deployment,” *arXiv preprint* arXiv:2504.13141, 2025.

[41] M. Xu *et al.*, “Auto-scaling approaches for microservice applications: A survey and taxonomy,” *arXiv preprint* arXiv:2507.17128, 2025.

[42] B. Jeong and Y.-S. Jeong, “Autoscaling techniques in cloud-native computing: A comprehensive survey,” *Computer Science Review*, Art. no. 100791, 2025, doi: 10.1016/j.cosrev.2025.100791.

[43] “Self-adaptation in microservice systems: Literature review informed classification taxonomy,” *Information and Software Technology*, Art. no. 108135, 2026, doi: 10.1016/j.infsof.2026.108135.

[44] “Kubernetes autoscaling: A comprehensive review on machine learning techniques,” *Preprints.org*, manuscript 202606.1094, 2026.

[45] G. Zhou, W. Tian, R. Buyya, R. Xue, and L. Song, “Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions,” *Artificial Intelligence Review*, vol. 57, no. 5, Art. no. 124, 2024, doi: 10.1007/s10462-024-10756-9.

[46] Kubernetes SIGs, “Metrics Server.” [Online]. Available: https://github.com/kubernetes-sigs/metrics-server

[47] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003, doi: 10.1109/MC.2003.1160055.

[48] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[49] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[50] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928–1937.

[51] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint* arXiv:1707.06347, 2017.

[52] J. von Kistowski *et al.*, "TeaStore: A micro-service reference application for benchmarking, modeling and resource management research," in *Proc. IEEE MASCOTS*, 2018, doi: 10.1109/MASCOTS.2018.00030.

[53] Y. Gan *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. ACM ASPLOS*, 2019, doi: 10.1145/3297858.3304013.

[54] Alibaba Group, "Alibaba Cluster Trace Program: Microservices trace dataset." [Online]. Available: https://github.com/alibaba/clusterdata

PLAGIARISM CHECK REPORT

[Paste the screenshot of the plagiarism check report here (similarity below 15%)]